IT5411
OPERATING SYSTEMS LABORATORY

A PRACTICAL RECORD

Submitted By

Yuvaraj K
2022506079
B. Tech Information Technology (4/8)

DEPARTMENT OF INFORMATION TECHNOLOGY
MADRAS INSTITUTE OF TECHNOLOGY
ANNA UNIVERSITY, CHENNAI – 600 044

February – May 2024

BONAFIDE CERTIFICATE

Name	**:** Yuvaraj K
Register No.	**:** 2022506079
Subject	**:** IT5411 Operating Systems Laboratory
Department	**:** Information Technology

Certified to the bonafide record of practical work done by Mr. Yuvaraj K in the IT5411 Operating Systems Laboratory during the period February 2024 to May 2024.

Date:

Dr D. Vivekanandan
STAFF-IN-CHARGE

Submitted for the Practical Examination held on: 23/05/2024

Internal Examiner External Examiner

IT5411 OS LABORATORY (R2019)

COURSE OBJECTIVES

To learn about the basic commands of operating systems.
To implement process synchronisation mechanisms in operating systems.
To learn various process management schemes in operating systems.
To practice with the important memory management mechanisms.
To implement the file allocation techniques.

COURSE OUTCOMES

CO1	Understand and implement basic services and functionalities of the operating system using system calls.
CO2	Use modern OS system calls and synchronisation libraries in software/hardware interfaces.
CO3	Understand the benefits of thread over process and implement synchronised programs using multithreading concepts.
CO4	Analyse various IPC techniques in the operating system.
CO5	Implement memory management schemes.
CO6	Simulate file allocation and organisation techniques.

CO-PO MATRIX

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12
CO1	3	3	2	2	3	2	2	3	2	3	2	2
CO2	3	2	2	2	3	2	2	3	2	3	2	2
CO3	2	1	2	2	3	2	2	2	2	3	2	2
CO4	3	1	2	2	3	2	2	2	3	3	3	2
CO5	2	1	3	3	3	2	2	2	3	3	3	2
CO6	2	1	2	3	3	2	2	2	3	3	3	2

DEPARTMENT OF INFORMATION TECHNOLOGY,
MIT CAMPUS, ANNA UNIVERSITY

EVALUATION RUBRICS FOR IT5411 OPERATING SYSTEMS LABORATORY

Method:

1. Continuous Assessment
2. One Lab Internal and
3. One End Semester Exam

S. No.	Method	Marks (100)
(A) Internal Marks (60)		
1	Continuous Assessment – (with Record)	50
2	Laboratory Examination (Internal)	25
		Total: (75) is converted to 60
(B) External Marks (40)		
3	End Semester Examination	25
		Total: (25) is converted to 40

(A) GRADING RUBRIC FOR INTERNAL EXAMINATION

I. RUBRIC USED FOR CONTINUOUS ASSESSMENT

Parameters	**Outstanding (20)**	**Excellent (18-19)**	**Good (16-17)**	**Average (13-15)**	**Poor (10-12)**	**Grading**
Observation – Preparedness, Basic implementation, Ability to adapt additional features and design standards **(40)**	100%	80-90%	60-70%	30-50%	10-20%	39/40
Record (10)	100%	80-90%	60-70%	30-50%	10-20%	/10
					Total (50)	**/50**

II. RUBRIC USED FOR CONTINUOUS ASSESSMENT

Parameters	**Outstanding (20)**	**Excellent (18-19)**	**Good (16-17)**	**Average (13-15)**	**Poor (10-12)**	**Grading**
Question Writeup – Problem Analysis and Design **(15)**	100%	80-90%	60-70%	30-50%	10-20%	/15
Execution **(10)**	Best	Good	Closer	Approximate	Irrelevant	/10
					Total (25)	**/25**

(B) GRADING RUBRIC FOR END SEMESTER EXAMINATION

II. RUBRIC USED FOR END SEMESTER EXAMINATION

Parameters	**Outstanding (20)**	**Excellent (18-19)**	**Good (16-17)**	**Average (13-15)**	**Poor (10-12)**	**Grading**
Question Writeup – Problem Analysis and Design **(15)**	100%	80-90%	60-70%	30-50%	10-20%	/15
Execution **(10)**	Best	Good	Closer	Approximate	Irrelevant	/10
					Total (25)	**/25**

Dr D. Vivekanandan
STAFF-IN-CHARGE

List of Experiments

Exp. No	Date	Name of the Experiment	Page	Signature
1	15/02/24	BASIC UNIX COMMANDS	1	
2	22/02/24	A) BASIC SHELL SCRIPTING	65	
	29/02/24	B) ADVANCED SHELL SCRIPTING	85	
3	07/03/24	PROCESS CONTROL SYSTEM CALLS	95	
4, 5	14/03/24	THREAD MANAGEMENT AND SYNCHRONISATION	101	
6	21/03/24	DEADLOCK AVOIDANCE USING SEMAPHORES	109	
7	28/03/24	SIMULATION OF PREEMPTIVE AND NON-PREEMPTIVE PROCESS SCHEDULING ALGORITHMS	117	
8	04/04/24	SIMULATION OF FILE ALLOCATION STRATEGIES	137	
9	18/04/24	INTER PROCESS COMMUNICATION USING PIPES	147	
10	18/04/24	INTER PROCESS COMMUNICATION USING FIFOs	153	
11	25/04/24	IMPLEMENTATION OF CPU SCHEDULING POLICIES	157	
12	02/05/24	IMPLEMENTATION OF MEMORY MANAGEMENT POLICIES	169	

Exp. No	1
Date	15/02/24

BASIC UNIX COMMANDS

This experiment maps to the following COs and POs.

CO1	Understand and implement basic services and functionalities of the operating system using system calls.
PO1	Engineering knowledge
PO5	Modern tool usage
PO9	Individual and team work

AIM

To study and implement the basic UNIX commands and the variations that come with them:

- File System Commands
- Directory Navigation Commands
- Hardware Information Commands
- Searching Commands
- User and Group Commands
- System Management and Information Commands
- File Permission Commands
- Shell Command Management
- Process Related Commands

(1) FILE SYSTEM COMMANDS

1) mkdir

Description	Make directories; create directories if they don't exist.
Syntax	mkdir [OPTION] DIRECTORY
Options	**-m:** set file mode (as in chmod) **-p:** make parent directories as needed **-v:** print a message for each created directory

COMMANDS

mkdir test1
mkdir -m 777 test2
mkdir -p test3/test4
mkdir -v test4

ls
ls test 3

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ mkdir test1
warspinix@Yuvaraj-Laptop:~/os/exp_01$ mkdir -m 777 test2
warspinix@Yuvaraj-Laptop:~/os/exp_01$ mkdir -p test3/test4
warspinix@Yuvaraj-Laptop:~/os/exp_01$ mkdir -v test4
mkdir: created directory 'test4'
warspinix@Yuvaraj-Laptop:~/os/exp_01$ ls
test1  test2  test3  test4
warspinix@Yuvaraj-Laptop:~/os/exp_01$ ls test3
test4
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

2) rmdir

Description	Remove empty directories.
Syntax	rmdir [OPTION] DIRECTORY
Options	**--ignore-fail-on-non-empty:** ignore each failure that's solely because a directory is non-empty **-p:** remove DIRECTORY and its ancestors; eg: *rmdir -p a/b/c* is the same as *rmdir a/b/c a/b a* **-v:** output a diagnostic for every directory processed

COMMANDS

rmdir test5
rmdir --ignore-fail-on-non-empty test5
ls
rmdir -p test3/test4
ls
rmdir -v test4
ls

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ rmdir test5
rmdir: failed to remove 'test5': No such file or directory
warspinix@Yuvaraj-Laptop:~/os/exp_01$ rmdir  --ignore-fail-on-non-empty test3
warspinix@Yuvaraj-Laptop:~/os/exp_01$ ls
test1  test2  test3  test4
warspinix@Yuvaraj-Laptop:~/os/exp_01$ rmdir -p test3/test4
warspinix@Yuvaraj-Laptop:~/os/exp_01$ ls
test1  test2  test4
warspinix@Yuvaraj-Laptop:~/os/exp_01$ rmdir -v test4
rmdir: removing directory, 'test4'
warspinix@Yuvaraj-Laptop:~/os/exp_01$ ls
test1  test2
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

3) touch

Description	Update the access and modification times of each file to the current time. A file argument that doesn't exist is created empty.
Syntax	touch [OPTION] FILE
Options	**-a:** change only the access time **-c:** do not create any files **-m:** change only the modification time

COMMANDS

touch f1.txt
ls

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ touch f1.txt
warspinix@Yuvaraj-Laptop:~/os/exp_01$ ls
f1.txt  test1  test2
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

4) rm

Description	Remove files or directories; removes each specified file. By default, it doesn't remove directories.
Syntax	rm [OPTION] FILE
Options	**-i:** prompt before every removal **-r:** remove directories and their contents recursively **-d:** remove empty directories

COMMANDS

rm -i f1.txt
ls
rm f1.txt
ls

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ rm -i f1.txt
rm: remove regular empty file 'f1.txt'? n
warspinix@Yuvaraj-Laptop:~/os/exp_01$ ls
f1.txt  test1  test2
warspinix@Yuvaraj-Laptop:~/os/exp_01$ rm f1.txt
warspinix@Yuvaraj-Laptop:~/os/exp_01$ ls
test1  test2
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

5) cp

Description	Copy source to destination, or to directory; copy files and directories.
Syntax	cp [OPTION] SOURCE DEST cp [OPTION] SOURCE DIRECTORY cp [OPTION] -t DIRECTORY SOURCE
Options	**-i:** prompt before overwrite **-l:** hard link files instead of copying **-s:** make symbolic links instead of copying **-t:** copy all source arguments into directory **-T:** treat DEST as a normal file

COMMANDS

touch f1.txt f2.txt
echo hello >> f1.txt
cp f1.txt f2.txt
cat f1.txt f2.txt
echo bye > f1.txt
cp -i f1.txt f2.txt
cp -t test2 f1.txt f2.txt
ls
ls test2
cat f1.txt f2.txt

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ touch f1.txt f2.txt
warspinix@Yuvaraj-Laptop:~/os/exp_01$ echo hello >> f1.txt
warspinix@Yuvaraj-Laptop:~/os/exp_01$ cp f1.txt f2.txt
warspinix@Yuvaraj-Laptop:~/os/exp_01$ cat f1.txt f2.txt
hello
hello
warspinix@Yuvaraj-Laptop:~/os/exp_01$ echo bye > f1.txt
warspinix@Yuvaraj-Laptop:~/os/exp_01$ cp -i f1.txt f2.txt
cp: overwrite 'f2.txt'? y
warspinix@Yuvaraj-Laptop:~/os/exp_01$ cp -t test2 f1.txt f2.txt
warspinix@Yuvaraj-Laptop:~/os/exp_01$ ls
f1.txt  f2.txt  test1  test2
warspinix@Yuvaraj-Laptop:~/os/exp_01$ ls test2
f1.txt  f2.txt
warspinix@Yuvaraj-Laptop:~/os/exp_01$ cat f1.txt f2.txt
bye
bye
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

6) mv

Description	Move/rename files; rename source to destination, or move source(s) to directory
Syntax	mv [OPTION] SOURCE DEST mv [OPTION] SOURCE DIRECTORY mv [OPTION] -t DIRECTORY SOURCE
Options	**-f:** don't prompt before overwriting **-i:** prompt before overwrite **-t:** move all source arguments into directory **-T:** treat DEST as a normal file

COMMANDS

mv f1.txt f2.txt
ls

touch f.txt
ls
mv f.txt test2
ls
ls test2

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ mv f1.txt f2.txt
warspinix@Yuvaraj-Laptop:~/os/exp_01$ ls
f2.txt  test1  test2
warspinix@Yuvaraj-Laptop:~/os/exp_01$ touch f.txt
warspinix@Yuvaraj-Laptop:~/os/exp_01$ ls
f.txt  f2.txt  test1  test2
warspinix@Yuvaraj-Laptop:~/os/exp_01$ mv f.txt test2
warspinix@Yuvaraj-Laptop:~/os/exp_01$ ls
f2.txt  test1  test2
warspinix@Yuvaraj-Laptop:~/os/exp_01$ ls test2
f.txt  f1.txt  f2.txt
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

7) ln

Description	Make links between files; create hard links by default, symbolic links with *--symbolic*
Syntax	ln [OPTION] -T TARGET LINK_NAME ln [OPTION] TARGET DIRECTORY
Options	**-f:** remove existing destination files **-i:** prompt whether to remove destinations **-s:** make symbolic links instead of hard links

COMMANDS

ln -s f.txt f3.txt
ls
ls -l

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ ln -s f.txt f3.txt
warspinix@Yuvaraj-Laptop:~/os/exp_01$ ls
f.txt  f2.txt  f3.txt  test1  test2
warspinix@Yuvaraj-Laptop:~/os/exp_01$ ls -l
total 12
-rw-r--r-- 1 warspinix warspinix    0 May  4 16:43 f.txt
-rw-r--r-- 1 warspinix warspinix    4 May  4 16:13 f2.txt
lrwxrwxrwx 1 warspinix warspinix    5 May  4 16:43 f3.txt -> f.txt
drwxr-xr-x 2 warspinix warspinix 4096 May  4 12:38 test1
drwxrwxrwx 2 warspinix warspinix 4096 May  4 16:33 test2
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

8) cat

Description	Concatenate file(s) to standard output. With no file, or when file is -, read standard input
Syntax	cat [OPTION] [FILE]
Options	**-n:** number all output lines **-b:** number non-empty output lines, overrides *-n*

COMMANDS

cat f2.txt
cat -n f2.txt

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ cat f2.txt
bye
warspinix@Yuvaraj-Laptop:~/os/exp_01$ cat -n f2.txt
     1  bye
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

9) vim

Description	A text editor used to edit all kinds of plain text. It is especially useful for editing programs.
Syntax	vim [OPTION] FILE vi [OPTION] FILE

COMMANDS

vi f2.txt

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ vi f2.txt
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

```
bye
// Pressing i to insert, i.e. edit
My name is
Yuvaraj.
This is an
example of
vi editor.
// Press esc and :wq to save and exit
```

10) nano

Description	Nano is a small and friendly editor. It copies the look and feel of Pico, but is free software, and implements several features that Pico lacks.
Syntax	nano [OPTION] FILE
Options	**-B:** when saving a file, backup the previous version, using the current filename suffixed with a tilde **-V:** show the current version number and exit

COMMANDS

nano f2.txt

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ nano f2.txt
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

```
  GNU nano 6.2                                    f2.txt *
bye
// Pressing i to insert, i.e. edit
My name is
Yuvaraj.
This is an
example of
vi editor.
// Press esc and :wq to save and exit
Now, I am in
nano editor.
This is different
from the vi/vim
editor.
// Press Ctrl+O to save
// Press Ctrl+X to exit

File Name to Write: f2.txt
^G Help          M-D DOS Format    M-A Append        M-B Backup File
^C Cancel        M-M Mac Format    M-P Prepend       ^T Browse
```

11) gedit

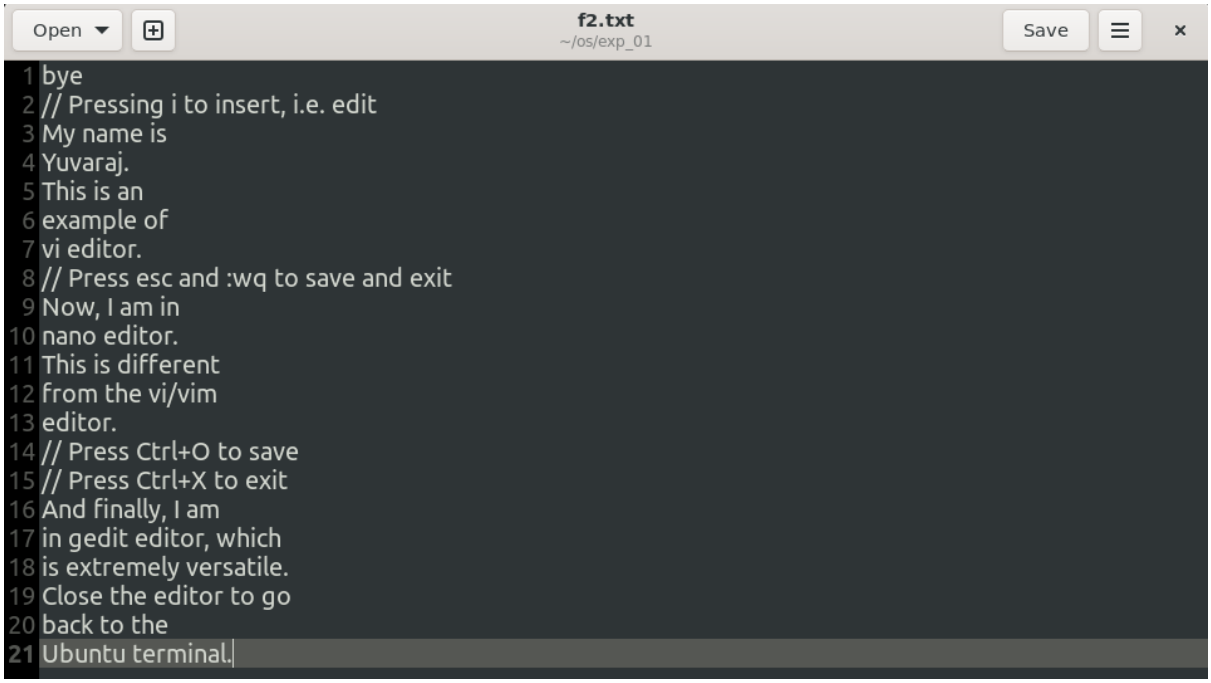
Description	Gedit is the official text editor of the GNOME desktop environment. It is a powerful general purpose text editor, and can be used to create and edit all kinds of text files.
Syntax	nano [OPTION] FILE
Options	**-s:** run gedit in standalone mode **-w:** open files and block the gedit process

COMMAND

gedit f2.txt

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ gedit f2.txt
```



12) head

Description	Print the first 10 lines of each FILE to standard output. With more than one FILE, precede each with a header giving the file name.
Syntax	head [OPTION] FILE
Options	**-n:** print the given no. of lines instead of first 10. **-v:** always print headers giving file names

COMMAND

head -n 7 f2.txt
head -v f2.txt

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ head -n 7 f2.txt
bye
// Pressing i to insert, i.e. edit
My name is
Yuvaraj.
This is an
example of
vi editor.
warspinix@Yuvaraj-Laptop:~/os/exp_01$ head -v f2.txt
==> f2.txt <==
bye
// Pressing i to insert, i.e. edit
My name is
Yuvaraj.
This is an
example of
vi editor.
// Press esc and :wq to save and exit
Now, I am in
nano editor.
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

13) tail

Description	Print the last 10 lines of each FILE to standard output. With more than one FILE, precede each with a header giving the file name.
Syntax	tail [OPTION] FILE
Options	**-n:** print the given no. of lines instead of first 10. **-v:** always print headers giving file names

COMMAND

tail -n 7 f2.txt
tail -v f2.txt

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ tail -n 7 f2.txt
// Press Ctrl+X to exit
And finally, I am
in gedit editor, which
is extremely versatile.
Close the editor to go
back to the
Ubuntu terminal.
warspinix@Yuvaraj-Laptop:~/os/exp_01$ tail -v f2.txt
==> f2.txt <==
from the vi/vim
editor.
// Press Ctrl+O to save
// Press Ctrl+X to exit
And finally, I am
in gedit editor, which
is extremely versatile.
Close the editor to go
back to the
Ubuntu terminal.
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

14) more

Description	A filter for paging through text on screenful at a time. This version is especially primitive.
Syntax	more [OPTION] FILE
Options	**-s:** squeeze multiple blank lines into one.

COMMANDS

man vi > f1.txt
more f1.txt

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ man vi > f1.txt
warspinix@Yuvaraj-Laptop:~/os/exp_01$ more f1.txt
VIM(1)           General Commands Manual           VIM(1)

NAME
       vim - Vi IMproved, a programmer's text editor

SYNOPSIS
       vim [options] [file ..]
       vim [options] -
       vim [options] -t tag
       vim [options] -q [errorfile]

       ex
       view
       gvim gview evim eview
       rvim rview rgvim rgview

DESCRIPTION
       Vim is a text editor that is upwards compatible to
       Vi.  It can be used to edit  all  kinds  of  plain
       text.   It  is  especially useful for editing pro-
       grams.

       There are a lot of enhancements  above  Vi:  multi
--More--(2%)
```

15) less

Description	Less is a program similar to more, but it has many more features. Less doesn’t have to read the entire input file before starting, so with large input files, it starts up faster than text editors like vi.
Syntax	less FILE
Options	**h or H:** display a summary of these commands **e:** forward one line **y:** backward one line

COMMAND

less f1.txt

OUTPUT

```
VIM(1)          General Commands Manual          VIM(1)

NAME
       vim - Vi IMproved, a programmer's text editor

SYNOPSIS
       vim [options] [file ..]
       vim [options] -
       vim [options] -t tag
       vim [options] -q [errorfile]

       ex
       view
       gvim gview evim eview
       rvim rview rgvim rgview

DESCRIPTION
       Vim is a text editor that is upwards compatible to
       Vi.  It can be used to edit  all  kinds  of  plain
       text.   It  is  especially useful for editing pro-
       grams.

       There are a lot of enhancements  above  Vi:  multi
       level  undo,  multi  windows  and  buffers, syntax
       highlighting, command line editing, filename  com-
       pletion,  on-line  help,  visual  selection, etc..
       See ":help vi_diff.txt" for a summary of the  dif-
       ferences between Vim and Vi.

       While  running  Vim  a lot of help can be obtained
       from the on-line help  system,  with  the  ":help"
       command.  See the ON-LINE HELP section below.

       Most  often  Vim  is started to edit a single file
       with the command

            vim file

       More generally Vim is started with:

:
```

16) gpg

Description	gpg is a tool to provide digital encryption and signing services using the OpenPGP standard.
Syntax	gpg [OPTIONS] FILE
Options	**-h:** print a usage message summarising the most useful command-line options **-c:** encrypt with a symmetric cipher using a passphrase **-d:** decrypt the file given on the command line

COMMANDS

gpg -c f2.txt
gpg -d f2.txt.gpg

OUTPUT

```
Enter passphrase

Passphrase: **************_________________________

      <OK>                              <Cancel>
```

```
Please re-enter this passphrase

Passphrase: **************_________________________

      <OK>                              <Cancel>
```

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ gpg -d f2.txt.gpg
gpg: AES256.CFB encrypted data
gpg: encrypted with 1 passphrase
bye
// Pressing i to insert, i.e. edit
My name is
Yuvaraj.
This is an
example of
vi editor.
// Press esc and :wq to save and exit
Now, I am in
nano editor.
This is different
from the vi/vim
editor.
// Press Ctrl+O to save
// Press Ctrl+X to exit
And finally, I am
in gedit editor, which
is extremely versatile.
Close the editor to go
back to the
Ubuntu terminal.
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

17) wc

Description	Print newline, word, and byte counts for each FILE, and a total line if more than one file is specified.
Syntax	wc [OPTIONS] FILE
Options	**-c:** print the byte count **-m:** print the character count **-l:** print the newline count **-w:** print the word count

COMMANDS

wc f1.txt
wc -c f1.txt
wc -m f1.txt
wc -l f1.txt
wc -w f1.txt

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ wc f1.txt
  545  2493 22023 f1.txt
warspinix@Yuvaraj-Laptop:~/os/exp_01$ wc -c f1.txt
22023 f1.txt
warspinix@Yuvaraj-Laptop:~/os/exp_01$ wc -m f1.txt
21917 f1.txt
warspinix@Yuvaraj-Laptop:~/os/exp_01$ wc -l f1.txt
545 f1.txt
warspinix@Yuvaraj-Laptop:~/os/exp_01$ wc -w f1.txt
2493 f1.txt
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

18) cut

Description	Print selected parts of lines from each FILE to standard output. With no FILE, or when FILE is -, read standard input.
Syntax	cut [OPTIONS] FILE
Options	**-b:** select only these bytes **-c:** select only these characters **-d:** use delimiter **-f:** select only these fields

COMMANDS

echo hello > f3.txt
echo bye >> f3.txt
cut -b 1,2 f3.txt
cut -c 1-2, 4-5 f3.txt
cut -d “e” -f 1 f3.txt

OPTION

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ echo hello > f3.txt
warspinix@Yuvaraj-Laptop:~/os/exp_01$ echo bye >> f3.txt
warspinix@Yuvaraj-Laptop:~/os/exp_01$ cut -b 1,2 f3.txt
he
by
warspinix@Yuvaraj-Laptop:~/os/exp_01$ cut -c 1-2,4-5 f3.txt
helo
by
warspinix@Yuvaraj-Laptop:~/os/exp_01$ cut -d "e" -f 1 f3.txt
h
by
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

19) shred

Description	Overwrite a file to hide its contents, and optionally delete it.
Syntax	shred [OPTIONS] FILE
Options	**-f:** change permissions to allow writing if necessary **-u:** deallocate and remove file after overwriting

COMMANDS

head f3.txt
shred f3.txt
head -n 5 f3.txt
rm f3.txt

OUTPUT

20) diff

Description	Compare files line by line.
Syntax	diff [OPTION] FILE1 FILE2
Options	**-q:** report only when files differ

COMMANDS

cat f.txt
cat f.txt > g.txt
echo “another hello” >> g.txt
cat g.txt
diff f.txt g.txt

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ cat f.txt
hello
bye
warspinix@Yuvaraj-Laptop:~/os/exp_01$ cat f.txt > g.txt
warspinix@Yuvaraj-Laptop:~/os/exp_01$ echo "another hello" >> g.txt
warspinix@Yuvaraj-Laptop:~/os/exp_01$ cat g.txt
hello
bye
another hello
warspinix@Yuvaraj-Laptop:~/os/exp_01$ diff f.txt g.txt
2a3
> another hello
warspinix@Yuvaraj-Laptop:~/os/exp_01$ |
```

21) source

Description	Read and execute commands from FILENAME in the current shell
Syntax	source FILENAME

COMMANDS

echo "echo hello" > f3.txt
cat f3.txt
source f3.txt

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ echo "echo hello" > f3.txt
warspinix@Yuvaraj-Laptop:~/os/exp_01$ cat f3.txt
echo hello
warspinix@Yuvaraj-Laptop:~/os/exp_01$ source f3.txt
hello
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

(2) DIRECTORY NAVIGATION COMMANDS

22) ls

Description	List files and directories in the current directory.
Syntax	ls [OPTION] [DIRECTORY]
Options	**-a:** do not ignore entries starting with **.**; show all files and directories, including hidden files **-l:** use long listing format

COMMANDS

ls
ls -l

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ ls
f.txt  f1.txt  f2.txt  f2.txt.gpg  f3.txt  g.txt  test1  test2
warspinix@Yuvaraj-Laptop:~/os/exp_01$ ls -l
total 52
-rw-r--r-- 1 warspinix warspinix    10 May  4 19:10 f.txt
-rw-r--r-- 1 warspinix warspinix 22023 May  4 17:56 f1.txt
-rw-r--r-- 1 warspinix warspinix   365 May  4 18:54 f2.txt
-rw-r--r-- 1 warspinix warspinix   303 May  4 18:48 f2.txt.gpg
-rw-r--r-- 1 warspinix warspinix    11 May 11 17:02 f3.txt
-rw-r--r-- 1 warspinix warspinix    24 May 11 16:58 g.txt
drwxr-xr-x 2 warspinix warspinix  4096 May  4 12:38 test1
drwxrwxrwx 2 warspinix warspinix  4096 May  4 16:33 test2
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

23) pwd

Description	Print the full filename of the current working directory.
Syntax	pwd [OPTION]
Options	**-L:** use PWD from environment, even if it contains symlinks **-P:** avoid all symlinks

COMMAND

pwd

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ pwd
/home/warspinix/os/exp_01
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

24) cd

Description	Change the shell working directory.
Syntax	cd [OPTIONS] DIRECTORY
Options	**~:** change to $HOME **..:** move up one directory level **-:** change to the previous directory

COMMANDS

cd ..
cd exp_01/test1
cd –
cd exp_01

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ cd ..
warspinix@Yuvaraj-Laptop:~/os$ cd exp_01/test1
warspinix@Yuvaraj-Laptop:~/os/exp_01/test1$ cd -
/home/warspinix/os
warspinix@Yuvaraj-Laptop:~/os$ cd exp_01
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

25) dirs

Description	Display directory stack; display the list of currently remembered directories,
Syntax	dirs [OPTIONS]
Options	**-p:** print the directory stack with one entry per line **-v:** print the directory stack with one entry per line prefixed with its position in the stack

COMMANDS

cd test1
dirs
dirs -v
cd ..

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ cd test1
warspinix@Yuvaraj-Laptop:~/os/exp_01/test1$ dirs
~/os/exp_01/test1
warspinix@Yuvaraj-Laptop:~/os/exp_01/test1$ dirs -v
 0  ~/os/exp_01/test1
warspinix@Yuvaraj-Laptop:~/os/exp_01/test1$ cd ..
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

(3) HARDWARE INFORMATION COMMANDS

26) lscpu

Description	Gathers CPU architecture information from sysfs, and any applicable architecture specific libraries.
Syntax	lscpu [OPTIONS]
Options	**-e:** display the CPU information in human readable format **-p:** optimise the command output for easy parsing

COMMAND

lscpu -e

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ lscpu -e
CPU SOCKET CORE L1d:L1i:L2:L3 ONLINE
  0      0    0 0:0:0:0          yes
  1      0    0 0:0:0:0          yes
  2      0    1 1:1:1:0          yes
  3      0    1 1:1:1:0          yes
  4      0    2 2:2:2:0          yes
  5      0    2 2:2:2:0          yes
  6      0    3 3:3:3:0          yes
  7      0    3 3:3:3:0          yes
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

27) lsblk

Description	Lists information about all available or the specified block devices.
Syntax	lsblk [OPTIONS]
Options	**-a:** also list empty devices and RAM disk devices **-D:** print information about the discarding capabilities for each device

COMMAND

lsblk

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ lsblk
NAME MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
sda    8:0    0 388.5M  1 disk
sdb    8:16   0     2G  0 disk [SWAP]
sdc    8:32   0     1T  0 disk /snap
                               /mnt/wslg/distro
                               /
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

28) lspci

Description	Utility for displaying information about PCI buses in the system and devices connected to them.
Syntax	lspci [OPTIONS]
Options	**-t:** show a tree-like diagram containing all buses, bridges, devices, and connections between them **-v:** be verbose and display detailed information about all devices

COMMAND

lspci -tv

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ lspci -tv
-+-[dfe0:00]---00.0  Red Hat, Inc. Virtio console
 +-[3da6:00]---00.0  Microsoft Corporation Device 008e
 +-[091d:00]---00.0  Red Hat, Inc. Virtio file system
 \-[0000:00]-
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

29) lsusb

Description	Utility for displaying information about USB buses in the system and the devices connected to them.
Syntax	lsusb [OPTIONS]
Options	**-t:** tells lsusb to dump the physical USB device hierarchy as a tree **-v:** tells lsusb to be verbose and display detailed information about the devices shown

COMMAND

lsusb -tv

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ lsusb -tv
/:  Bus 02.Port 1: Dev 1, Class=root_hub, Driver=vhci_hcd/8p, 5000M
    ID 1d6b:0003 Linux Foundation 3.0 root hub
/:  Bus 01.Port 1: Dev 1, Class=root_hub, Driver=vhci_hcd/8p, 480M
    ID 1d6b:0002 Linux Foundation 2.0 root hub
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

30) lshw

Description	A small tool to extract detailed information on the hardware configuration of the machine.
Syntax	lshw [OPTIONS]
Options	**-x:** launch the X11 GUI (if available) **-html:** outputs the device tree as an HTML page **-short:** outputs the device tree showing hardware paths, very much like the output of HP-UX's ioscan

COMMAND

lshw -short

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ sudo lshw -short
[sudo] password for warspinix:
H/W path    Device    Class      Description
==========================================
                      system     Computer
/0                    bus        Motherboard
/0/1                  memory     8064MiB System memory
/0/2                  processor  11th Gen Intel(R) Core(TM) i7-1165G7 @ 2.80
/0/3                  generic    Virtio file system
/0/3/0                generic    Virtual I/O device
/0/4                  display    Microsoft Corporation
/0/0                  storage    Virtio console
/0/0/0                generic    Virtual I/O device
/0/5                  system     PnP device PNP0b00
/0/6        scsi0     storage
/0/6/0.0.0  /dev/sda  volume     388MiB Virtual Disk
/0/6/0.0.1  /dev/sdb  volume     2GiB Virtual Disk
/0/6/0.0.2  /dev/sdc  volume     1TiB Virtual Disk
/1          usb1      bus        USB/IP Virtual Host Controller
/2          usb2      bus        USB/IP Virtual Host Controller
/3          eth0      network    Ethernet interface
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

31) free

Description	Display the amount of free and used memory in the system.
Syntax	free [OPTIONS]
Options	**-b:** display the amount of memory in bytes **-k:** display the amount of memory in kibibytes (default) **-m:** display the amount of memory in mebibytes **-g:** display the amount of memory in gibibytes **-h:** show all output fields automatically scaled to shortest 3-digit unit and display the units of print out

COMMAND

free -h

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ free -h
               total        used        free      shared  buff/cache   available
Mem:           7.6Gi       514Mi       6.3Gi       3.0Mi       850Mi       6.9Gi
Swap:          2.0Gi          0B       2.0Gi
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

(4) SEARCHING COMMANDS

32) find

Description	Search for files in a directory hierarchy.
Syntax	find [OPTIONS] expression
Options	**-L:** follow symbolic links

COMMAND

find t*

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ find t*
test1
test2
test2/f.txt
test2/f1.txt
test2/f2.txt
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

33) grep

Description	Searches for patterns in each file.
Syntax	grep [OPTIONS] PATTERN FILE
Options	**-E:** interpret patterns as extended regular expressions **-i:** ignore case distinctions in patterns and input data, so that characters that differ only in case match each other **-v:** invert the sense of matching, to select non-matching lines **-c:** print a count of matching lines **-o:** print only the matched (non-empty) parts of a matching line, with each such part on a separate output line

COMMANDS

grep is f2.txt
grep -o is f2.txt
grep -i this f2.txt
grep -c is f2.txt

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ grep is f2.txt
My name is
This is an
This is different
is extremely versatile.
warspinix@Yuvaraj-Laptop:~/os/exp_01$ grep -o is f2.txt
is
is
is
is
is
is
warspinix@Yuvaraj-Laptop:~/os/exp_01$ grep -i this f2.txt
This is an
This is different
warspinix@Yuvaraj-Laptop:~/os/exp_01$ grep -c is f2.txt
4
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

34) locate

Description	Reads one or more databases prepared by updated and writes the file names matching at least one of the PATTERNS
Syntax	locate [OPTION] PATTERN
Options	**-c:** write the no. of matching entries only **-i:** ignore case distinctions when matching patterns **-r:** search for a basic regex pattern

COMMANDS

locate recovery-mode
locate -c recovery-mode

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ locate recovery-mode
/usr/lib/recovery-mode
/usr/lib/recovery-mode/l10n.sh
/usr/lib/recovery-mode/options
/usr/lib/recovery-mode/recovery-menu
/usr/lib/recovery-mode/options/apt-snapshots
/usr/lib/recovery-mode/options/clean
/usr/lib/recovery-mode/options/dpkg
/usr/lib/recovery-mode/options/failsafeX
/usr/lib/recovery-mode/options/fsck
/usr/lib/recovery-mode/options/grub
/usr/lib/recovery-mode/options/network
/usr/lib/recovery-mode/options/root
/usr/lib/recovery-mode/options/system-summary
warspinix@Yuvaraj-Laptop:~/os/exp_01$ locate -c recovery-mode
13
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

35) which

Description	Shows the full path of (shell) commands.
Syntax	which [OPTIONS] programname
Options	**-a:** print all matching executables in PATH, not just the first

COMMAND

which sed

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ which sed
/usr/bin/sed
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

36) whereis

Description	Locate the binary, source, and manual file pages for a command.
Syntax	whereis [OPTION] name
Options	**-b:** search for binaries **-m:** search for manuals **-s:** search for sources

COMMAND

whereis -b sed

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ whereis -b sed
sed: /usr/bin/sed
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

37) awk/gawk

Description	Pattern scanning and processing language.
Syntax	awk [OPTIONS] FILE
Options	**-f:** read the awk program source from the file, instead of from the first command line argument **-c:** run in compatibility mode

COMMANDS

awk '/is/ {print $1}' f2.txt
awk '/is/ {print $1, $2}' f2.txt
awk '/is/ {print $1, $2, $3}' f2.txt

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ awk '/is/ {print $1}' f2.txt
My
This
This
is
warspinix@Yuvaraj-Laptop:~/os/exp_01$ awk '/is/ {print $1, $2}' f2.txt
My name
This is
This is
is extremely
warspinix@Yuvaraj-Laptop:~/os/exp_01$ awk '/is/ {print $1, $2, $3}' f2.txt
My name is
This is an
This is different
is extremely versatile.
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

38) sed

Description	Stream editor for filtering and transforming text.
Syntax	sed [OPTION] 's/[old]/[new]/OPTION' file
Options	**-e:** add the script to the commands to be executed **-i:** edit files in place

COMMAND

sed s/is/was/g f2.txt

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ sed s/is/was/g f2.txt
bye
// Pressing i to insert, i.e. edit
My name was
Yuvaraj.
Thwas was an
example of
vi editor.
// Press esc and :wq to save and exit
Now, I am in
nano editor.
Thwas was different
from the vi/vim
editor.
// Press Ctrl+O to save
// Press Ctrl+X to exit
And finally, I am
in gedit editor, which
was extremely versatile.
Close the editor to go
back to the
Ubuntu terminal.
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

(5) USER AND GROUP COMMANDS

39) id

Description	Print real and effective user and group IDs.
Syntax	id [OPTION] [USER]
Options	**-u:** print only the effective user ID

COMMAND

id -u warspinix

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ id -u warspinix
1000
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

40) last

Description	Show a listing of last logged in users.
Syntax	last [OPTIONS] [USERNAME]
Options	**-d:** for non-local logins **-i:** displays host's IP number instead of the name

COMMANDS

last warspinix > f4.txt
head f4.txt

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ last warspinix > f4.txt
warspinix@Yuvaraj-Laptop:~/os/exp_01$ head f4.txt
warspini pts/1                         Sat May 11 16:51    gone - no logout
warspini pts/1                         Thu May  9 09:48 - crash (2+07:03)
warspini pts/1                         Thu May  9 08:25 - crash  (01:23)
warspini pts/1                         Thu May  9 05:38 - crash  (02:46)
warspini pts/1                         Wed May  8 21:27 - crash  (08:11)
warspini pts/1                         Wed May  8 13:07 - crash  (08:19)
warspini pts/1                         Wed May  8 10:29 - crash  (02:37)
warspini pts/1                         Wed May  8 10:28 - crash  (00:01)
warspini pts/1                         Wed May  8 08:35 - crash  (01:52)
warspini pts/1                         Tue May  7 04:31 - crash (1+04:04)
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

41) who

Description	Show who is logged on.
Syntax	who [OPTION] [FILE | ARG1 ARG2]
Options	**-b:** time of last system boot **-m:** only hostname and user associated with stdin

COMMANDS

who
who -b

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ who
warspinix pts/1        2024-05-11 16:51
warspinix@Yuvaraj-Laptop:~/os/exp_01$ who -b
         system boot  2024-05-11 16:51
warspinix@Yuvaraj-Laptop:~/os/exp_01$ |
```

(6) SYSTEM MANAGEMENT COMMANDS

42) date

Description	Print or display the current time.
Syntax	date [OPTION] [FORMAT]
Options	**-d:** display the time displayed by string, not now **-s:** set time described by string **-u:** print or set Coordinated Universal Time (UTC)

COMMANDS

date
date -u
date -d '21:21:21'
date -d '2004-09-21 11:30:21'
date -d 'next Tuesday'

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ date
Sat May 11 19:33:06 IST 2024
warspinix@Yuvaraj-Laptop:~/os/exp_01$ date -u
Sat May 11 14:03:09 UTC 2024
warspinix@Yuvaraj-Laptop:~/os/exp_01$ date -d '21:21:21'
Sat May 11 21:21:21 IST 2024
warspinix@Yuvaraj-Laptop:~/os/exp_01$ date -d '2004-09-21 11:30:21'
Tue Sep 21 11:30:21 IST 2004
warspinix@Yuvaraj-Laptop:~/os/exp_01$ date -d 'next Tuesday'
Tue May 14 00:00:00 IST 2024
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

43) uname

Description	Print system information.
Syntax	uname [OPTION]
Options	**-a:** print all information **-r:** print the kernel release

COMMANDS

uname
uname -r

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ uname
Linux
warspinix@Yuvaraj-Laptop:~/os/exp_01$ uname -r
5.15.146.1-microsoft-standard-WSL2
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

44) cal

Description	Displays a simple calendar.
Syntax	cal [OPTION] [[day] [month] [year]]
Options	**-y:** display a calendar for the specified year **-3:** display the previous, current, and next month **ncal :** vertical layout

COMMANDS

cal
cal -3 9 2024
ncal 9 2024

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ cal
      May 2024
Su Mo Tu We Th Fr Sa
          1  2  3  4
 5  6  7  8  9 10 11
12 13 14 15 16 17 18
19 20 21 22 23 24 25
26 27 28 29 30 31

warspinix@Yuvaraj-Laptop:~/os/exp_01$ cal -3 9 2024
    August 2024          September 2024         October 2024
Su Mo Tu We Th Fr Sa  Su Mo Tu We Th Fr Sa  Su Mo Tu We Th Fr Sa
             1  2  3   1  2  3  4  5  6  7         1  2  3  4  5
 4  5  6  7  8  9 10   8  9 10 11 12 13 14   6  7  8  9 10 11 12
11 12 13 14 15 16 17  15 16 17 18 19 20 21  13 14 15 16 17 18 19
18 19 20 21 22 23 24  22 23 24 25 26 27 28  20 21 22 23 24 25 26
25 26 27 28 29 30 31  29 30                 27 28 29 30 31

warspinix@Yuvaraj-Laptop:~/os/exp_01$ ncal 9 2024
    September 2024
Su  1  8 15 22 29
Mo  2  9 16 23 30
Tu  3 10 17 24
We  4 11 18 25
Th  5 12 19 26
Fr  6 13 20 27
Sa  7 14 21 28
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

45) uptime

Description	Gives a one-line display of the following information: - current time - how long the system has been running - how many users are currently logged on - system load averages for the past 1, 5, and 15 minutes
Syntax	uptime [OPTION]
Options	**-p:** show uptime in pretty format **-s:** system up since, in yyyy-mm-dd HH:MM:SS format

COMMANDS

uptime
uptime -p
uptime -s

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ uptime
 19:51:20 up  2:43,  1 user,  load average: 0.16, 0.03, 0.01
warspinix@Yuvaraj-Laptop:~/os/exp_01$ uptime -p
up 2 hours, 43 minutes
warspinix@Yuvaraj-Laptop:~/os/exp_01$ uptime -s
2024-05-11 17:07:55
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

46) hostname

Description	Used to display the system's DNS name, and to display or set its hostname or NIS domain name
Syntax	hostname [OPTIONS]
Options	**-d:** display the name of the DNS domain **-f:** display the Fully Qualified Domain Name **-i:** show the IP address of the system

COMMAND

hostname

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ hostname
Yuvaraj-Laptop
warspinix@Yuvaraj-Laptop:~/os/exp_01$ hostname -i
127.0.1.1
warspinix@Yuvaraj-Laptop:~/os/exp_01$ |
```

47) w

Description	Show who is logged on and what they are doing. The header shows - uptime information - information about the users currently on the machine - the users' processes
Syntax	w [OPTIONS] [USER]
Options	**-h:** don't print the header **-s:** use the short format **user:** show information about the specified user only

COMMAND

w
w -s
w -h

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ w
 19:59:26 up  2:51,  1 user,  load average: 0.04, 0.02, 0.00
USER     TTY      FROM             LOGIN@   IDLE   JCPU   PCPU WHAT
warspini pts/1    -                16:51    3:07m  0.02s  0.02s -bash
warspinix@Yuvaraj-Laptop:~/os/exp_01$ w -s
 19:59:28 up  2:51,  1 user,  load average: 0.04, 0.02, 0.00
USER     TTY      FROM              IDLE WHAT
warspini pts/1    -                 3:07m -bash
warspinix@Yuvaraj-Laptop:~/os/exp_01$ w -h
warspini pts/1    -                16:51    3:07m  0.02s  0.02s -bash
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

48) whoami

Description	Print the user name associated with the current effective user ID.
Syntax	whoami [OPTION]

COMMAND

whoami

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ whoami
warspinix
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

49) finger

Description	Displays information about the system users.
Syntax	finger [OPTIONS] [USERNAME]

COMMAND

finger

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ finger
Login       Name        Tty      Idle  Login Time   Office     Office Phone
warspinix              *pts/1    3:11  May 11 16:51
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

(7) FILE PERMISSION COMMANDS

50) chmod

Description	Change the access permissions of the named files; can't change the permissions of symbolic links.
Syntax	chmod [OPTIONS] MODE FILENAME
Options	**-v:** output a diagnostic for every file processed **-f:** suppress most error messages

COMMANDS

ls -l f.txt
chmod 755 f.txt
ls -l f.txt

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ ls -l f.txt
-rw-r--r-- 1 warspinix warspinix 10 May  4 19:10 f.txt
warspinix@Yuvaraj-Laptop:~/os/exp_01$ chmod 755 f.txt
warspinix@Yuvaraj-Laptop:~/os/exp_01$ ls -l f.txt
-rwxr-xr-x 1 warspinix warspinix 10 May  4 19:10 f.txt
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

(8) SHELL COMMAND MANAGEMENT

51) alias

Description	Define or display aliases.
Syntax	alias [OPTION] [name[=value]]
Options	**-p:** print all defined aliases in a reusable format

COMMANDS

alias rm='rm -i'
alias rm
rm f3.txt
unalias rm
alias rm

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ alias rm='rm -i'
warspinix@Yuvaraj-Laptop:~/os/exp_01$ alias rm
alias rm='rm -i'
warspinix@Yuvaraj-Laptop:~/os/exp_01$ rm f3.txt
rm: remove regular file 'f3.txt'? n
warspinix@Yuvaraj-Laptop:~/os/exp_01$ unalias rm
warspinix@Yuvaraj-Laptop:~/os/exp_01$ alias rm
-bash: alias: rm: not found
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

52) man

Description	Linux system's manual pager, an interface to the system reference manuals
Syntax	man [OPTIONS] COMMAND
Options	**-f:** equivalent to whatis **-k:** equivalent to apropos

COMMAND

man -f grep

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ man -f grep
grep (1)             - print lines that match patterns
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

53) help

Description	Display information about built-in commands/shell features.
Syntax	help [OPTION] [COMMAND]
Options	**-d:** output short description for each topic

COMMAND

help -d alias

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ help -d alias
alias - Define or display aliases.
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

54) info

Description	To read documentation in *Info* format.
Syntax	info [OPTION] [COMMAND]
Options	**-a:** use all matching manuals **-w:** print physical location of *Info* file

COMMANDS

info -a sed
info -w sed

OUTPUT

```
Node: File names matching 'sed'

Info File Index
***************

File names that match 'sed':

* Menu:

*     1: (/usr/share/info/sed.info.gz).
*     2: (*manpages*)sed.
```

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ info -a sed
warspinix@Yuvaraj-Laptop:~/os/exp_01$ info -w sed
/usr/share/info/sed.info.gz
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

55) history

Description	Keeps track of all inputs/commands read from the user.
Syntax	history

COMMANDS

history > f5.txt
tail -n 5 f5.txt

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ history > f5.txt
warspinix@Yuvaraj-Laptop:~/os/exp_01$ tail -n 5 f5.txt
 2007  ks
 2008  clear
 2009  ls
 2010  clear
 2011  history > f5.txt
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

56) apropos

Description	Search the manual page names and descriptions.
Syntax	apropos [OPTION] keyword
Options	**-v:** print verbose warning messages **-e:** each keyword will be exactly matched against the page names and descriptions

COMMAND

apropos who

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ apropos who
w (1)                - Show who is logged on and what they are doing.
who (1)              - show who is logged on
whoami (1)           - print effective userid
warspinix@Yuvaraj-Laptop:~/os/exp_01$ |
```

57) type

Description	Display information about command type.
Syntax	type [OPTION] name
Options	**-a:** display all locations containing the executable **-f:** suppress shell function lookup

COMMANDS

type mv
type help

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ type mv
mv is /usr/bin/mv
warspinix@Yuvaraj-Laptop:~/os/exp_01$ type help
help is a shell builtin
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

(9) PROCESS RELATED COMMANDS

58) ps

Description	Displays information about a selection of the active processes.
Syntax	ps [OPTION]
Options	**-d:** select all processes except session leaders **-a:** select all processes except both session leaders and processes not associated with a terminal

COMMAND

ps

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ ps
    PID TTY          TIME CMD
    369 pts/0    00:00:00 bash
   3351 pts/0    00:00:00 ps
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

59) coproc

Description	Creates a coprocess which is executed asynchronously, with the standard input and standard output connected via a pipe.
Syntax	coproc [NAME] command

COMMANDS

coproc man vim
ps

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ coproc man vim
[1] 3892
warspinix@Yuvaraj-Laptop:~/os/exp_01$ ps
    PID TTY          TIME CMD
    369 pts/0    00:00:00 bash
   3909 pts/0    00:00:00 ps
[1]+  Done                    coproc COPROC man vim
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

60) sleep

Description	Delay for a specified amount of time; pause for given no. of seconds.
Syntax	sleep NUMBER[SUFFIX]

COMMAND

sleep 100

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ sleep 100


^C
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

61) jobs

Description	Display the list of active jobs.
Syntax	jobs [OPTION]
Options	**-l:** lists process IDs in addition to the normal information **-p:** list process IDs only

COMMAND

coproc sleep 20
jobs
jobs -p
jobs

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ coproc sleep 20
[1] 4754
warspinix@Yuvaraj-Laptop:~/os/exp_01$ jobs
[1]+  Running                 coproc COPROC sleep 20 &
warspinix@Yuvaraj-Laptop:~/os/exp_01$ jobs -p
4754
warspinix@Yuvaraj-Laptop:~/os/exp_01$ jobs
[1]+  Done                    coproc COPROC sleep 20
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

62) fg

Description	Place the jobs identified by JOB_SPEC in the foreground, making it the current job.
Syntax	fg [JOB_SPEC]

COMMAND

coproc sleep 30
fg

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ coproc sleep 30
[1] 5246
warspinix@Yuvaraj-Laptop:~/os/exp_01$ fg
coproc COPROC sleep 30
^C
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

63) bg

Description	Place the jobs identified by each JOB_SPEC in the background; if the JOB_SPEC is not present, the shell's notion of the current job is used.
Syntax	bg [JOB_SPEC]

COMMAND

sleep 50
bg
ps

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ sleep 50
^Z
[1]+  Stopped                 sleep 50
warspinix@Yuvaraj-Laptop:~/os/exp_01$ bg
[1]+ sleep 50 &
warspinix@Yuvaraj-Laptop:~/os/exp_01$ ps
    PID TTY          TIME CMD
    369 pts/0    00:00:00 bash
   5831 pts/0    00:00:00 sleep
   5850 pts/0    00:00:00 ps
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

64) kill

Description	Kills/terminates processes under the given process ID; sends a signal to processes
Syntax	kill [SIGNAL] PID
Options	**15:** graceful termination of process **1:** hang up/disconnect a process **2:** interrupt a process **9:** immediately terminates a process **19:** stop a process temporarily

COMMAND

coproc sleep 60
kill -9 6561
ps

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_01$ coproc sleep 60
[1] 6561
warspinix@Yuvaraj-Laptop:~/os/exp_01$ kill -9 6561
warspinix@Yuvaraj-Laptop:~/os/exp_01$ ps
    PID TTY          TIME CMD
    369 pts/0    00:00:00 bash
   6593 pts/0    00:00:00 ps
[1]+  Killed                  coproc COPROC sleep 60
warspinix@Yuvaraj-Laptop:~/os/exp_01$
```

Evaluation Criteria	Observation	Record
Ability for problem definition and realisation	**8/10**	**/10**
Ability to design and analyse	**8/10**	**/10**
Ability to implement and validate	**8/10**	**/10**

RESULT

The basic UNIX commands were successfully studied and implemented.

Exp. No	2A
Date	22/02/24

BASIC SHELL SCRIPTING

This experiment maps to the following COs and POs.

CO1	Understand and implement basic services and functionalities of the operating system using system calls.
CO2	Use modern OS system calls and synchronisation libraries in software/hardware interfaces.
PO1	Engineering knowledge
PO2	Problem analysis
PO5	Modern tool usage
PO9	Individual and team work

AIM

To study, explore, and implement basic programs in bash shell scripting.

(1) ARITHMETIC OPERATIONS

```
# !/usr/bin/bash
printf "Enter Number 1: "
read x
printf "Enter Number 2: "
reda y
(( sum = x + y ))
(( difference = x - y ))
(( product = x * y ))
(( quotient = x / y ))
(( remainder = x % y ))
(( power = x ** y ))
echo "Sum = $sum"
echo "Difference = $difference"
echo "Product = $product"
echo "Quotient = $quotient"
echo "Remainder = $remainder"
echo "Power = $power"
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_02a$ bash 1_arithmeticOperations.sh
Enter Number 1: 10
Enter Number 2: 3
Sum = 13
Difference = 7
Product = 30
Quotient = 3
Remainder = 1
Power = 1000
warspinix@Yuvaraj-Laptop:~/os/exp_02a$
```

(2) FACTORIAL

```
# !/usr/bin/bash
printf "Enter a Number: "
read n

f=1

# For Loop

for (( i=1;i<=n;i++ )) do
	(( f*=i ))
done
echo "Using for loop, factorial of $n is $f."

# While Loop

i=1
f=1

while [[ $i -le $n ]]; do
	(( f*=i ))
	(( i++ ))
done
echo "Using while loop, factorial of $n is $f."
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_02a$ bash 2_factorial.sh
Enter a Number: 7
Using for loop, factorial of 7 is 5040.
Using while loop, factorial of 7 is 5040.
warspinix@Yuvaraj-Laptop:~/os/exp_02a$
```

(3) MULTIPLICATION TABLE

```
# !/usr/bin/bash
printf "Enter a Number: "
read x
printf "No. of Terms: "
read n
printf "\nMultiplication Table of $x\n"
for (( i=1;i<=n;i++ )) do
        (( p = x*i ))
        echo "$x * $i = $p"
done
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_02a$ bash 3_multiplicationTable.sh
Enter a Number: 3
No. of Terms: 7

Multiplication Table of 3
3 * 1 = 3
3 * 2 = 6
3 * 3 = 9
3 * 4 = 12
3 * 5 = 15
3 * 6 = 18
3 * 7 = 21
warspinix@Yuvaraj-Laptop:~/os/exp_02a$
```

(4) ARMSTRONG NUMBER

```
# !/usr/bin/bash
printf "Enter a Number: "
read n
digits=0
s=0
x=$n
while [ $x -gt 0 ]; do
	(( digits += 1 ))
	(( x = x/10 ))
done

x=$n

while [ $x -gt 0 ]; do
	(( t = x%10 ))
	(( s += t**digits ))
	(( x = x/10 ))
done

if [ $s -eq $n ]; then
	echo "$n is an Armstrong Number."
else
	echo "$n is not an Armstrong Number."
Fi
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_02a$ bash 4_armstrongNumber.sh
Enter a Number: 28
28 is not an Armstrong Number.
warspinix@Yuvaraj-Laptop:~/os/exp_02a$ bash 4_armstrongNumber.sh
Enter a Number: 370
370 is an Armstrong Number.
warspinix@Yuvaraj-Laptop:~/os/exp_02a$
```

(5) STRING STATISTICS

```
# !/usr/bin/bash

printf "Enter a String: "
read string
vowelCount=0
consonantCount=0
numCount=0
spaceCount=0
specialCount=0
wordCount=1

if [ -n "$string" ]; then
	(( l = ${#string} ))
	for (( i=0; i<l; i++ )) do
		f=(${string:i:1})
		s=(${f,})
		if [[ $s == *[aeiou]* ]]; then
			(( vowelCount++ ))
		elif [[ $s == *[bcdfghjklmnpqrstvwxyz]* ]]; then
			(( consonantCount++ ))
		elif [[ $s == *[0123456789]* ]]; then
			(( numCount++ ))
		elif [[ $s == "" ]]; then
			(( spaceCount++ ))
			(( wordCount++ ))
		else
			(( specialCount++ ))
		fi
	done
	echo ""
	echo "Using Traditional Methods..."
	echo "Length of the String = $l"
	echo "No. of Words = $wordCount"
	echo "Vowels = $vowelCount"
	echo "Consonants = $consonantCount"
	echo "Digits = $numCount"
	echo "Spaces = $spaceCount"
	echo "Special Characters = $specialCount"
```

```
    echo ""
    echo "Character and Word Count using Linux Commands..."
    words=$(echo -n "$string" | wc -w)
    characters=$(echo -n "$string" | wc -c)
    echo "No. of Words = $words"
    echo "No. of Characters = $characters"
else
    echo "String is Empty."
fi
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_02a$ bash 5_stringCheck.sh
Enter a String: Hi! This is 1 string overall.

Using Traditional Methods...
Length of the String = 29
No. of Words = 6
Vowels = 7
Consonants = 14
Digits = 1
Spaces = 5
Special Characters = 2

Character and Word Count using Linux Commands...
No. of Words = 6
No. of Characters = 29
warspinix@Yuvaraj-Laptop:~/os/exp_02a$
```

(6) PRIME NUMBERS

```
# !/usr/bin/bash
printf "Enter a Number: "
read n
echo ""
echo "Prime Numbers upto $n"
for (( i=2; i<=n; i++ )) do
	flag=1
	for (( j=2; j<i; j++ )) do
		(( x = i%j ))
		if [[ $x -eq 0 ]]; then
			flag=0
		fi
	done
	if [[ $flag -eq 1 ]]; then
		printf "$i "
	fi
done
echo ""
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_02a$ bash 6_primeNumbers.sh
Enter a Number: 30

Prime Numbers upto 30
2 3 5 7 11 13 17 19 23 29
warspinix@Yuvaraj-Laptop:~/os/exp_02a$
```

(7) FIBONACCI NUMBERS

```
# !/usr/bin/bash

printf "Enter a Number: "
read n
printf "\nFirst $n Fibonacci Numbers\n"
a=0
b=1
printf "1) $a\n2) $b\n"
for (( i=3; i<=n; i++ )) do
	(( c = a + b ))
	echo "$i) $c"
	(( a = b ))
	(( b = c ))
Done
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_02a$ bash 7_fibonacciNumbers.sh
Enter a Number: 15

First 15 Fibonacci Numbers
1) 0
2) 1
3) 1
4) 2
5) 3
6) 5
7) 8
8) 13
9) 21
10) 34
11) 55
12) 89
13) 144
14) 233
15) 377
warspinix@Yuvaraj-Laptop:~/os/exp_02a$
```

(8) LCM AND GCD

```
printf "Enter Number 1: "
read x
printf "Enter Number 2: "
read y
if [[ $x -gt $y ]]; then
	(( small = y ))
	(( large = x ))
else
	(( small = x ))
	(( large = y ))
fi
while true; do
	(( t1 = large%x ))
	(( t2 = large%y ))
	if [[ $t1 -eq 0 && $t2 -eq 0 ]]; then
		(( lcm = large ))
		break
	fi
	(( large++ ))
done
for (( i=1; i<=small; i++ )) do
	(( t1 = x%i ))
	(( t2 = y%i ))
	if [[ $t1 -eq 0 && $t2 -eq 0 ]]; then
		(( gcd = i ))
	fi
done
echo "LCM of $x and $y is $lcm."
echo "GCD of $x and $y is $gcd."
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_02a$ bash 8_LCM_and_GCD.sh
Enter Number 1: 4
Enter Number 2: 10
LCM of 4 and 10 is 20.
GCD of 4 and 10 is 2.
warspinix@Yuvaraj-Laptop:~/os/exp_02a$
```

(9) FACTORS OF GIVEN NUMBER

```
# !/usr/bin/bash
printf "Enter a Number: "
read n

printf "\nFactors of $n\n"
for (( i=1; i<=n; i++ )) do
	(( t = n%i ))
	if [[ $t -eq 0 ]]; then
		printf "$i "
	fi
done
echo ""
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_02a$ bash 9_factors.sh
Enter a Number: 21

Factors of 21
1 3 7 21
warspinix@Yuvaraj-Laptop:~/os/exp_02a$
```

(10) NEON NUMBER

```
# !/usr/bin/bash
: <<END_COMMENT
Program to Check if a given number is a Neon Number
A neon number is a number
where the sum of digits of
square of the number is
equal to the number.
Eg: Consider the number 9.
9*9=81 (square of the number)
8+1=9 (sum of the digits of the square)
It is equal to the original number.
So, 9 is a neon number.
END_COMMENT
printf "Enter a Number: "
read n
(( s = n**2 ))
sod=0
while [ $s -gt 0 ]; do
	(( sod += s%10 ))
	(( s /= 10 ))
done
if [ $sod -eq $n ]; then
	echo "$n is a Neon Number."
else
	echo "$n is not a Neon Number."
fi
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_02a$ bash 10_neonNumber.sh
Enter a Number: 9
9 is a Neon Number.
warspinix@Yuvaraj-Laptop:~/os/exp_02a$ bash 10_neonNumber.sh
Enter a Number: 99
99 is not a Neon Number.
warspinix@Yuvaraj-Laptop:~/os/exp_02a$
```

(11) LARGEST AND SMALLEST AMONG N

```
printf "No. of Elements: "
read n
printf "\nNumber 1: "
read x
(( numbers[0] = x ))
(( largest = x ))
(( smallest = x ))
for (( i=2; i<=n; i++ )) do
	printf "Number $i: "
	read x
	(( numbers[i-1] = x ))
	if [ $x -gt $largest ]; then
		(( largest = x ))
	fi
	if [ $x -lt $smallest ]; then
		(( smallest = x ))
	fi
done
printf "\nAll Elements\n"
for (( i=0; i<n; i++ )) do
	(( x = numbers[i] ))
	printf "$x "
done
printf "\nLargest = $largest"
printf "\nSmallest = $smallest\n"
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_02a$ bash 11_minmaxAmongN.sh
No. of Elements: 3

Number 1: 7
Number 2: 3
Number 3: 21

All Elements
7 3 21
Largest = 21
Smallest = 3
warspinix@Yuvaraj-Laptop:~/os/exp_02a$
```

(12) AGE CALCULATION

```
# !/usr/bin/bash
printf "Enter your birth date\n"
printf "Date (DD): "
read bd
printf "Month (MM): "
read bm
printf "Year (YYYY): "
read by

td=$(date +%d)
tm=$(date +%m)
ty=$(date +%Y)

(( age = ty - by ))

if [ $tm -lt $bm ]; then
        (( age-- ))
fi
if [ $tm -eq $bm ]; then
       if [ $td -lt $bd ]; then
        (( age-- ))
       fi
fi

printf "\nAge = $age\n"
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_02a$ bash 12_ageCalculation.sh
Enter your birth date
Date (DD): 07
Month (MM): 05
Year (YYYY): 2005

Age = 18
warspinix@Yuvaraj-Laptop:~/os/exp_02a$
```

(13) PATTERN PROGRAMS

```
# !/usr/bin/bash

printf "No. of Rows: "
read n

printf "\nPattern 1\n"
for (( i=1; i<=n; i++ )) do
	for (( j=1; j<=i; j++ )) do
		printf "* "
	done
	printf "\n"
done

printf "\nPattern 2\n"
for (( i=1; i<=n; i++ )) do
	for (( j=1; j<=i; j++ )) do
		printf "$j "
	done
	printf "\n"
done

printf "\nPattern 3\n"
for (( i=1; i<=n; i++ )) do
	for (( j=i; j>=1; j-- )) do
		printf "$j "
	done
	printf "\n"
done
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_02a$ bash 13_patterns.sh
No. of Rows: 7

Pattern 1
*
* *
* * *
* * * *
* * * * *
* * * * * *
* * * * * * *

Pattern 2
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
1 2 3 4 5 6
1 2 3 4 5 6 7

Pattern 3
1
2 1
3 2 1
4 3 2 1
5 4 3 2 1
6 5 4 3 2 1
7 6 5 4 3 2 1
warspinix@Yuvaraj-Laptop:~/os/exp_02a$
```

(14) COLOURING USING ANSI ESCAPE CODES

```
# !//usr/bin/bash
clear
echo -e "\e[1mHello There"      # bold
echo -e "\e[5mBlink"            # blink
echo -e "\e[0mHello There"      # back to normal
echo -e "\e[31mHello There"     # red
echo -e "\e[32mHello There"     # green
echo -e "\e[33mHello There"     # yellow
echo -e "\e[34mHello There"     # blue
echo -e "\e[35mHello There"     # magenta
echo -e "\e[36mHello There"     # cyan
echo -e "\e[0mHello There"      # back to normal
echo -e "\e[41mHello There"     # red
echo -e "\e[42mHello There"     # green
echo -e "\e[43mHello There"     # yellow
echo -e "\e[44mHello There"     # blue
echo -e "\e[45mHello There"     # magenta
echo -e "\e[46mHello There"     # cyan
echo -e "\e[0mHello There"      # back to normal
```

OUTPUT

```
Hello There
Blink
Hello There
Hello There
Hello There
Hello There
Hello There
Hello There
Hello There
Hello There
Hello There
Hello There
Hello There
Hello There
Hello There
Hello There
Hello There
warspinix@Yuvaraj-Laptop:~/os/exp_02a$
```

(15) FILE HANDLING

```
# !/usr/bin/bash
printf "File Name: "
read file
printf "\nContent to be Written: "
read contents
echo $contents > $file
printf "\nContents of the File:\n"
cat $file
printf "\nContent to be Overwritten: "
read contents
echo $contents > $file
printf "\nContents of the File:\n"
cat $file
printf "\nContent to be Appended: "
read contents
echo $contents >> $file
printf "\nContents of the File:\n"
cat $file
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_02a$ bash 15_fileHandling.sh
File Name: 15_file.txt

Content to be Written: hello

Contents of the File:
hello

Content to be Overwritten: bye

Contents of the File:
bye

Content to be Appended: hello

Contents of the File:
bye
hello
warspinix@Yuvaraj-Laptop:~/os/exp_02a$
```

Evaluation Criteria	Observation	Record
Ability for problem definition and realisation	**10/10**	**/10**
Ability to design and analyse	**10/10**	**/10**
Ability to implement and validate	**10/10**	**/10**

RESULT

Basic programs were successfully implemented in the bash environment.

Exp. No	2B
Date	29/02/24

ADVANCED SHELL SCRIPTING

This experiment maps to the following COs and POs.

CO1	Understand and implement basic services and functionalities of the operating system using system calls.
CO2	Use modern OS system calls and synchronisation libraries in software/hardware interfaces.
PO1	Engineering knowledge
PO2	Problem analysis
PO5	Modern tool usage
PO9	Individual and team work

AIM

To utilise a combination of UNIX commands and basic bash shell scripting in order to implement and solve advanced concepts in the bash environment.

(1) FILE ROTATION

Given *n* files, contents of the 1st file should go into the 2nd file, contents of the 2nd file should go into the 3rd file, …, and the contents of the *n*th file should go into the 1st file.

SOURCE CODE

```
# !/usr/bin/bash

printf "Total No. of Files: "
read n

for (( i=0; i<n; i++ )) do
	(( fno = i+1 ))
	printf "File $fno: "
	read x
	if ! [ -f "$x" ]; then
	  echo "File does not exist."
	  exit
	fi
	a[$i]=$x
done

(( last = n-1 ))
temp=${a[$last]}
lastContent=$(cat "$temp")

for (( i=n-1; i>0; i-- )) do
	sourcefile="${a[$((i-1))]}"
	destinationfile="${a[$i]}"
	content=$(cat "$sourcefile")
	echo "$content" > "$destinationfile"
done

firstfile="${a[0]}"
echo "$lastContent" > "$firstfile"

echo "Contents of the given $n files have been successfully rotated."
```

CONTENTS OF FILES BEFORE ROTATION

```
warspinix@Yuvaraj-Laptop:~/os/exp_02b$ cat f1
This is the first file.
File 1 contents.
This f1 file is the
first of many files
that are used for
the second experiment.
warspinix@Yuvaraj-Laptop:~/os/exp_02b$ cat f2
This is the second file.
File 2 contents.
This f2 file is the
second of many files
that are used for
the second experiment.
warspinix@Yuvaraj-Laptop:~/os/exp_02b$ cat f3
This is the third file.
File 3 contents.
This f3 file is the
third of many files
that are used for
the second experiment.
warspinix@Yuvaraj-Laptop:~/os/exp_02b$
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_02b$ bash mfc.sh
Total No. of Files: 3
File 1: f1
File 2: f2
File 3: f3
Contents of the given 3 files have been successfully rotated.
warspinix@Yuvaraj-Laptop:~/os/exp_02b$
```

CONTENTS OF FILES AFTER ROTATION

```
warspinix@Yuvaraj-Laptop:~/os/exp_02b$ cat f1
This is the third file.
File 3 contents.
This f3 file is the
third of many files
that are used for
the second experiment.
warspinix@Yuvaraj-Laptop:~/os/exp_02b$ cat f2
This is the first file.
File 1 contents.
This f1 file is the
first of many files
that are used for
the second experiment.
warspinix@Yuvaraj-Laptop:~/os/exp_02b$ cat f3
This is the second file.
File 2 contents.
This f2 file is the
second of many files
that are used for
the second experiment.
warspinix@Yuvaraj-Laptop:~/os/exp_02b$
```

(2) ROTATION AND REPLACEMENT OF SPECIFIC LINE RANGE

Given *n* files and a specific range of lines, said contents of the 1st file should go into the 2nd file, contents of the 2nd file should go into the 3rd file, …, and the contents of the *n*th file should go into the 1st file. There is not only rotation in this case, but also a specific replacement based on the provided input.

SOURCE CODE

```
# !/usr/bin/bash

printf "Total No. of Files: "
read n
printf "Lower Limit: "
read l
printf "Upper Limit: "
read u

for (( i=0; i<n; i++ )) do
	(( fno = i+1 ))
	printf "File $fno: "
	read x
	if ! [ -f "$x" ]; then
	  echo "File does not exist."
	  exit
	fi
	a[$i]=$x
done

(( last = n-1 ))
lastfile=${a[$last]}
lastContent=$(cat "$lastfile")
firstfile=${a[0]}

t="tempfile"
o=$(touch "$t")
```

```
for (( i=n-1; i>0; i-- )) do
	sourcefile="${a[$((i-1))]}"
	destinationfile="${a[$i]}"
	nl=$(wc -l "$destinationfile" |  cut -d ' ' -f 1)
	if [ $l -eq 1 ]; then
		p1=$(sed -n "${l},${u}p" "$sourcefile")
		(( x = u+1 ))
		p2=$(sed -n "${x},\$p" "$destinationfile")
		echo "$p1" > "$t"
		echo "$p2" >> "$t"
	elif [ $u -eq $nl ]; then
		(( x = l-1 ))
		p1=$(sed -n "1,${x}p" "$destinationfile")
		p2=$(sed -n "${l},${u}p" "$sourcefile")
		echo "$p1" > "$t"
		echo "$p2" >> "$t"
	else
		(( x = l-1 ))
		p1=$(sed -n "1,${x}p" "$destinationfile")
		p2=$(sed -n "${l},${u}p" "$sourcefile")
		(( y = u+1 ))
		p3=$(sed -n "${y},\$p" "$destinationfile")
		echo "$p1" > "$t"
		echo "$p2" >> "$t"
		echo "$p3" >> "$t"
	fi
	c=$(cat "$t")
	echo "$c" > "$destinationfile"
done

v="tempfile2"
o=$(touch "$v")
echo "$lastContent" > $v
nl=$(wc -l "$firstfile" |  cut -d ' ' -f 1)

if [ $l -eq 1 ]; then
	p1=$(sed -n "${l},${u}p" "$v")
	(( x = u+1 ))
	p2=$(sed -n "${x},\$p" "$firstfile")
	echo "$p1" > "$t"
```

```
        echo "$p2" >> "$t"

elif [ $u -eq "$nl" ]; then
        (( x = l-1 ))
        p1=$(sed -n "1,${x}p" "$firstfile")
        p2=$(sed -n "${l},${u}p" "$v")
        echo "$p1" > "$t"
        echo "$p2" >> "$t"

else
        (( x = l-1 ))
        p1=$(sed -n "1,${x}p" "$firstfile")
        p2=$(sed -n "${l},${u}p" "$v")
        (( y = u+1 ))
        p3=$(sed -n "${y},\$p" "$firstfile")
        echo "$p1" > "$t"
        echo "$p2" >> "$t"
        echo "$p3" >> "$t"

fi

c=$(cat "$t")
echo "$c" > "$firstfile"
printf "Contents of the $n files have successfully been rotated.\n"
r1=$(rm "$t")
r2=$(rm "$v")
```

CONTENTS OF FILES BEFORE ROTATION AND REPLACEMENT

```
warspinix@Yuvaraj-Laptop:~/os/exp_02b$ cat a1
File 1 Line 1
File 1 Line 2
File 1 Line 3
File 1 Line 4
File 1 Line 5
File 1 Line 6
File 1 Line 7
File 1 Line 8
File 1 Line 9
File 1 Line 10
File 1 Line 11
File 1 Line 12
warspinix@Yuvaraj-Laptop:~/os/exp_02b$ cat a2
File 2 Line 1
File 2 Line 2
File 2 Line 3
File 2 Line 4
File 2 Line 5
File 2 Line 6
File 2 Line 7
File 2 Line 8
File 2 Line 9
File 2 Line 10
File 2 Line 11
File 2 Line 12
warspinix@Yuvaraj-Laptop:~/os/exp_02b$ cat a3
File 3 Line 1
File 3 Line 2
File 3 Line 3
File 3 Line 4
File 3 Line 5
File 3 Line 6
File 3 Line 7
File 3 Line 8
File 3 Line 9
File 3 Line 10
File 3 Line 11
File 3 Line 12
File 3 Line 13
warspinix@Yuvaraj-Laptop:~/os/exp_02b$
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_02b$ bash mfcl.sh
Total No. of Files: 3
Lower Limit: 3
Upper Limit: 7
File 1: a1
File 2: a2
File 3: a3
Contents of the 3 files have successfully been rotated.
warspinix@Yuvaraj-Laptop:~/os/exp_02b$
```

CONTENTS OF FILES AFTER ROTATION AND REPLACEMENT

```
warspinix@Yuvaraj-Laptop:~/os/exp_02b$ cat a1
File 1 Line 1
File 1 Line 2
File 3 Line 3
File 3 Line 4
File 3 Line 5
File 3 Line 6
File 3 Line 7
File 1 Line 8
File 1 Line 9
File 1 Line 10
File 1 Line 11
File 1 Line 12
warspinix@Yuvaraj-Laptop:~/os/exp_02b$ cat a2
File 2 Line 1
File 2 Line 2
File 1 Line 3
File 1 Line 4
File 1 Line 5
File 1 Line 6
File 1 Line 7
File 2 Line 8
File 2 Line 9
File 2 Line 10
File 2 Line 11
File 2 Line 12
warspinix@Yuvaraj-Laptop:~/os/exp_02b$ cat a3
File 3 Line 1
File 3 Line 2
File 2 Line 3
File 2 Line 4
File 2 Line 5
File 2 Line 6
File 2 Line 7
File 3 Line 8
File 3 Line 9
File 3 Line 10
File 3 Line 11
File 3 Line 12
File 3 Line 13
warspinix@Yuvaraj-Laptop:~/os/exp_02b$
```

Evaluation Criteria	Observation	Record
Ability for problem definition and realisation	**10/10**	**/10**
Ability to design and analyse	**10/10**	**/10**
Ability to implement and validate	**10/10**	**/10**

RESULT

More advanced problems were successfully solved by combining UNIX commands and shell scripting ideas in the bash environment.

Exp. No	3
Date	07/03/24

PROCESS CONTROL SYSTEM CALLS

This experiment maps to the following COs and POs.

CO1	Understand and implement basic services and functionalities of the operating system using system calls.
CO2	Use modern OS system calls and synchronisation libraries in software/hardware interfaces.
CO3	Understand the benefits of thread over process and implement synchronised programs using multithreading concepts.
PO1	Engineering knowledge
PO2	Problem analysis
PO3	Design/development of solutions
PO5	Modern tool usage
PO9	Individual and team work
PO10	Communication

AIM

To implement process control system calls such as fork(), wait(), and exit() in a divide and conquer based problem and enable communication between parent and child processes.

PROCESS CONTROL

System calls are essential for the proper functioning of an operating system, as they provide a standardised way for programs to access system resources. One of the many services provided by system calls is the creation and management of processes.

There are a variety of different system calls made during process control that deal with the creation, waiting, and termination of processes.

- fork() is the function used to cause a *split*, creating a child process to which control is passed on.
- wait() is used in order to make the parent process wait for its child to return to it; it can continue its execution only after the return of its child.
- exit() is used to exit a particular process and return to its parent. If the process calling the exit() function is the parent, the program ends.

CHOSEN PROBLEM: BINARY SEARCH TREE

Insertion and search in a binary search tree are implemented using process control system calls.

When an element is being inserted into an empty binary search tree, the parent process does the insertion by itself. When subsequent elements are inserted, based on whether the element goes to the left subtree or the right subtree, child processes are accordingly invoked, and these child processes insert the element at its correct position. At each level, the number of child processes grows by a power of 2. Each process has its own process ID, by which it is identified.

It is similar in the case of searching in the binary search tree. The parent process compares the given element with the root node. If the element is lesser than the root, the parent invokes its left child, and if the element is greater, the parent invokes its right child. This goes on recursively until the element is found (OR) a NULL node is reached, i.e., the element is not found. In either case, the result is subsequently communicated back to the respective parent processes.

ALGORITHM

INPUT: *Binary Search Tree, data to be inserted*

OUTPUT*: Binary Search Tree with new data inserted at the correct position by child processes that are created whenever the data moves to a left or right subtree.*

BST-INSERT (root, data)

```
if root is NULL
        return new Node (data)
if data < root→data
        pid left = fork()
        if left
                root = BST-INSERT (root→left, data)
                exit()
         if parent
                wait (left, status, 0)
                if exited (status)
                        result = exit status
                        display left_pid, parent_pid details
else if data > root→data
        pid right = fork()
        if right
                root = BST-INSERT (root→right, data)
                exit()
        if parent
                wait (right, status, 0)
                if exited (status)
                        result = exit status
                        display right_pid, parent_pid details
return root
```

INPUT: *Binary Search Tree, data to be searched*
OUTPUT: *1 if the data is found in the Binary Search Tree, 0 otherwise*

BST-SEARCH (root, data)

```
if root is NULL
        return 0
if root→data == data
        return 1
if data < root→data
        pid left = fork()
        if child
                result = BST-SEARCH (root→left, data)
                exit (result)
        if parent
                wait (left, status, 0)
                if exited (status)
                        result = exit status
                        display left_pid, parent_pid details
                        if result == 1
                                return successful search
                        else
                                return failed search
else if data > root→data
        pid right = fork()
        if child
                result = BST-SEARCH (root→right, data)
                exit (result)
        if parent
                wait (right, status, 0)
                if exited (status)
                        result = exit status
                        display right_pid, parent_pid details
                        return result
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_03$ cc bstprocess.c
warspinix@Yuvaraj-Laptop:~/os/exp_03$ ./a.out
Manually insert elements into the binary search tree.
Enter each element followed by Enter, type -1 to finish.
10
Parent Process with Process ID 6382 inserting 10.
Process with Process ID 6382 has created a node with value 10.
5
Parent Process with Process ID 6382 inserting 5.
Process with Process ID 6382 is calling its left child.
Child Process with Process ID 6388 inserting 5 to the left of 10.
Process with Process ID 6388 has created a node with value 5.
15
Parent Process with Process ID 6388 inserting 15.
Process with Process ID 6388 is calling its right child.
Child Process with Process ID 6399 inserting 15 to the right of 10.
Process with Process ID 6399 has created a node with value 15.
3
Parent Process with Process ID 6399 inserting 3.
Process with Process ID 6399 is calling its left child.
Child Process with Process ID 6408 inserting 3 to the left of 10.
Process with Process ID 6408 is calling its left child.
Child Process with Process ID 6409 inserting 3 to the left of 5.
Process with Process ID 6409 has created a node with value 3.
-1
Value to be Searched: 3
Parent Process with Process ID 6409 searching for key 3.
Child Process with Process ID 6427 searching for key 3 in the left subtree of 10.
Child Process with Process ID 6428 searching for key 3 in the left subtree of 5.
Process with Process ID 6428 searching for key 3 in the root with value of 3.
Child Process with Process ID 6428 found the key 3 and has returned to its parent with Process ID 6427.
Child Process with Process ID 6427 found the key 3 and has returned to its parent with Process ID 6409.
Parent Process with Process ID 6409 found the key 3 in the binary search tree.

Inorder Traversal
3 5 10 15
```

Evaluation Criteria	Observation	Record
Ability for problem definition and realisation	**10/10**	**/10**
Ability to design and analyse	**10/10**	**/10**
Ability to implement and validate	**10/10**	**/10**

RESULT

Process creation and management were successfully implemented using process control system calls for the binary search tree insertion and search problems.

Exp. No	4, 5
Date	14/03/24

THREAD MANAGEMENT AND SYNCHRONISATION

This experiment maps to the following COs and POs.

CO2	Use modern OS system calls and synchronisation libraries in software/hardware interfaces.
CO3	Understand the benefits of thread over process and implement synchronised programs using multithreading concepts.
PO1	Engineering knowledge
PO2	Problem analysis
PO3	Design/development of solutions
PO5	Modern tool usage
PO9	Individual and team work
PO10	Communication

AIM

To work with, implement, and solve any synchronisation problem in a multithreading execution environment with the help of the POSIX thread model.

THREAD MANAGEMENT

A thread is a small instance running within a process. It can also be considered as a lightweight process. Just like processes, threads aren't independent. They share the same properties as regular processes, like an ID – which can be called a thread ID instead of a process ID – a program counter, and a particular register set along with stack space.

There are four basic thread management operations: *thread creation, thread termination, thread join,* and *thread yield.*

Multiple threads can be created and executed concurrently. The creating thread is generally called the *parent thread,* and the created thread is called the *child thread.*

In most cases, threads aren't run forever. In general, when the assigned task of a thread completes, the thread may be terminated. Moreover, if the parent thread terminates, all of its child threads terminate as well. But usually, parent threads are supposed to terminate after the termination of all of its child threads. This is because if a parent thread runs faster and terminates earlier than its child threads do, the child threads won't be able to access the resources owned by the parent threads and won't be able to perform the operations that they're supposed to, thus creating an inconsistent state.

This problem is solved by thread join. A thread can execute a thread join to wait until the other thread terminates. When a thread join is executed, only one thread is executed, and all the others are suspended. Once this thread is terminated, the others can resume.

There is another issue that arises in this case – the particular thread that is running might be eating up the CPU, preventing anything else that could potentially be executed from running. In this case, the thread has to be politely made to rest so that the CPU can be used by other threads. This is achieved by thread yield. When a thread executes a thread yield, the executing thread is suspended and the CPU is given to some other runnable thread. The thread will wait until the CPU becomes available again.

THREAD SYNCHRONISATION

Thread synchronisation is the concurrent execution of two or more threads that share critical resources. Threads should be properly synchronised in order to avoid conflicts in the usage of critical resources. Otherwise, conflicts may arise when parallel-running threads attempt to modify a common variable at the same time.

Synchronisation of threads can be achieved through various means. One well-known method is through mutex locks. Mutex is short for Mutual Exclusion. It is a type of lockable object that can be owned by exactly one thread at a time. Only the thread that acquired a lock can release the lock on a mutex. When the mutex is locked, any attempt to acquire the lock will fail or block, even if that attempt is done by the same thread.

POSIX THREAD

The C and C++ languages provide a standard pthread API for all kinds of thread related functions. It is also known as the POSIX thread, and allows users to implement multithreading for simultaneous processes to flow.

Function	Description
pthread_create()	Create a new thread.
pthread_exit()	Terminate a thread.
pthread_self()	Get the thread ID of the current thread.
pthread_equal()	Compare two threads. If they are the same, a non-zero value is returned, otherwise zero is returned.
pthread_cancel()	Send a cancellation request to a thread.
pthread_detach()	Detach a thread; a detached thread doesn’t require a thread to join on terminating. The resources of the thread are automatically released after terminating if the thread is detached.
pthread_join()	Wait for thread termination.

CHOSEN PROBLEM: COFFEE BREAK

There are 3 people who are on a coffee break, and each one of them has an infinite amount of one of the following three ingredients required to make coffee:

- coffee beans
- milk
- sugar

There is a coffee machine that supplies any 2 of these ingredients, say every 2 minutes. But what happens here is that, when 2 of these ingredients are made available, any one of the 3 people can use them, but only one of them can use both.

Eg: If the machine provides milk and sugar, the one who has the coffee beans can make the coffee. However, if the other two take on of these two ingredients, no one will be able to drink coffee, causing a deadlock.

So, when a particular set of ingredients are ready, the corresponding person is signalled, and the other 2 are made to wait, thereby avoiding deadlock.

ALGORITHM

INPUT: *'n' people waiting for their coffee break*
OUTPUT: *'n' threads simulating the 'n' people, managed and synchronised using a mutex lock – at every step, the person whose ingredients are available is called to use the coffee machine, while the others are made to wait.*

COFFEE-BREAK (person, m)

```
thread tid = self() → person
tid → wait
mutex_lock(m)
tid → using the coffee machine
tid → finished making coffee
mutex_unlock(m)
exit (NULL)
```

INITIALISE (n)

```
pthread_mutex m
n = no. of people
pthread threads[n]
string array people[n]
pthread_init (m, NULL)
for i=1 to n
        pthread_create (threads[i], COFFEE-BREAK, people[i])
for i=1 to n
        pthread_join (threads[i])
pthread_destroy (m)
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_04$ cc coffee.c
warspinix@Yuvaraj-Laptop:~/os/exp_04$ ./a.out
Krishnaa (ID: 139793613510208) is waiting for coffee.
Yuvaraj (ID: 139793596724800) is waiting for coffee.
Aashin (ID: 139793588332096) is waiting for coffee.
Kabilesh (ID: 139793605117504) is waiting for coffee.
BK (ID: 139793579939392) is waiting for coffee.

Aashin (ID: 139793588332096) is using the coffee machine.
Aashin (ID: 139793588332096) has finished making coffee.

Krishnaa (ID: 139793613510208) is using the coffee machine.
Krishnaa (ID: 139793613510208) has finished making coffee.

BK (ID: 139793579939392) is using the coffee machine.
BK (ID: 139793579939392) has finished making coffee.

Kabilesh (ID: 139793605117504) is using the coffee machine.
Kabilesh (ID: 139793605117504) has finished making coffee.

Yuvaraj (ID: 139793596724800) is using the coffee machine.
Yuvaraj (ID: 139793596724800) has finished making coffee.

All coffee breaks are over.
warspinix@Yuvaraj-Laptop:~/os/exp_04$
```

Evaluation Criteria	Observation	Record
Ability for problem definition and realisation	**10/10**	**/10**
Ability to design and analyse	**10/10**	**/10**
Ability to implement and validate	**10/10**	**/10**

RESULT

Thread management and synchronisation were successfully implemented and achieved for the coffee break synchronisation problem using the POSIX thread model.

Exp. No	6
Date	21/03/24

DEADLOCK AVOIDANCE USING SEMAPHORES

This experiment maps to the following COs and POs.

CO2	Use modern OS system calls and synchronisation libraries in software/hardware interfaces.
CO3	Understand the benefits of thread over process and implement synchronised programs using multithreading concepts.
PO1	Engineering knowledge
PO2	Problem analysis
PO3	Design/development of solutions
PO5	Modern tool usage
PO9	Individual and team work
PO10	Communication

AIM

To understand cooperative processes, deadlock conditions and recovery, deadlock avoidance and race condition, and choose a popular real world process synchronisation problem to solve using semaphores.

COOPERATIVE PROCESS

They are processes that depend on other processes. Cooperative processes interact with each other by sharing resource such as CPU, data, memory, and\or I/O devices. They share resources with other processes in the system by directly sharing a logical space, which is both code and data, or by sharing data through files or messages. They are essentially dependent processes.

DEADLOCK

Deadlock refers to a situation where one or more processes are not able to proceed because each is waiting for the other to release some resource. It is a state where a group of processes become stuck in a way that they can't make any progress.

CONDITIONS FAVOURING DEADLOCK

Mutual Exclusion	Only one process can use a resource at any given time, i.e., the resources are non-shareable.
Hold and Wait	A process is holding at least one resource at a time, and is waiting to acquire other resources held by some other process or processes.
No Pre-emption	The resource can be released by a process voluntarily, i.e., after execution of the process.
Circular Wait	A set of processes waiting for each other in a circular fashion. For example, let's say there are a set of processes $\{P_0, P_1, P_2, P_3\}$ such that P_0 depends on P_1, P_1 depends on P_2, P_2 depends on P_3, and P_3 depends on P_0. This creates a circular relation between all these processes, and they have to wait forever to be executed.

RECOVERY FROM DEADLOCK

It refers to the set of techniques and algorithms designed to detect, resolve, or mitigate deadlock situations. These methods ensure that the system can continue processing tasks efficiently without being trapped in an eternal standstill.

There's no mechanism implemented by the OS to avoid or prevent deadlocks – the system therefore assumes that a deadlock will undoubtedly occur. The OS periodically checks the system any deadlocks in an effort to break them. It will use various recovery techniques to restore the system if it encounters any deadlocks.

When a Deadlock Detection Algorithm determines that a deadlock has occurred in the system, they system must recover from that deadlock.

There are multiple ways to eliminate deadlocks.

RESOURCE PREEMPTION

Here, we pre-empt some resources from processes and give those resources to other processes. This method raises three issues.

Selecting a Victim	We must determine which resources and which processes are to be pre-empted such that the cost is also minimised.
Rollback	We must determine what should be done with the process from which resources are pre-empted. One simple idea is total rollback. That means aborting the process and restarting it all over again.
Starvation	In a system, it may happen that the same process is always picked as a victim. As a result, that process will never complete its designated task. This situation is called starvation, and must be avoided. One solution is that a process must be picked as a victim only a finite number of times.

ROLLBACK

In database systems, rolling back is a common technique for breaking deadlocks. When using this technique, the system reverses the transactions of the involved processes to a time before the deadlock. The system must keep a log of all transactions and the system's conditions at various points in time in order to use this method. The transactions can then be rolled back to the initial state and executed again by the system. This approach may result in significant delays in the transactions' execution and data loss.

DEADLOCK AVOIDANCE

A deadlock avoidance policy grants a resource request only if it can establish that granting the request can't lead to a deadlock either immediately or in the future.

The kernel lacks detailed knowledge about the future behaviour of processes, so it can't accurately predict deadlocks. To facilitate deadlock avoidance under these conditions, it uses a conservative approach.

Each process declares the maximum number of resource units of each class that it may require. The kernel permits a process to request these resource units in stages, i.e., a few resource units at a time, subject to the maximum number declared by it, and uses a worst-case analysis technique to check for the availability of future deadlocks. A request is granted only if there's no possibility of deadlocks; otherwise, it remains pending until it can be granted. This approach is conservative, because a process may complete its operation without requiring the maximum number of units declared by it.

RACE CONDITION

A race condition is an undesirable situation which occurs when the system attempts to perform 2 or more operations at the same time, but because of the nature of the system, the operations must be done in the proper sequence to be done correctly.

SEMAPHORES

Semaphores are a very relaxed type of lockable object that consists of a pre-defined maximum count, and a current count. A thread takes ownership of a semaphore using the *wait* operation, also referred to as *decrementing* the semaphore. Ownership of the semaphore is released using the *signal* operation, also referred to as *incrementing* the semaphore. While mutex ownership is tied very tightly to a thread, and only the thread that acquired the lock on a mutex can release it, semaphore ownership is far more relaxed. Any thread can signal a semaphore at any time, whether or not that thread has previously waited for the semaphore. Semaphores can be binary, wherein it has a maximum count of 1. On the other hand, counting semaphores can take any value up to the limit that is defined initially.

CHOSEN PROBLEM: COFFEE BREAK

There are 3 people who are on a coffee break, and each one of them has an infinite amount of one of the following three ingredients required to make coffee:

- coffee beans
- milk
- sugar

There is a coffee machine that supplies any 2 of these ingredients, say every 2 minutes. But what happens here is that, when 2 of these ingredients are made available, any one of the 3 people can use them, but only one of them can use both.

Eg: If the machine provides milk and sugar, the one who has the coffee beans can make the coffee. However, if the other two take on of these two ingredients, no one will be able to drink coffee, causing a deadlock.

So, when a particular set of ingredients are ready, the corresponding person is signalled, and the other 2 are made to wait, thereby avoiding deadlock.

ALGORITHM

INPUT: *'n' people waiting for their coffee break*
OUTPUT: *'n' threads simulating the 'n' people, managed and synchronised using a binary semaphore – at every step, the person whose ingredients are available is called to use the coffee machine, while the others are made to wait.*

COFFEE-BREAK (person, semaphore)

```
thread tid = self() → person
tid → wait
wait (semaphore)
getValue (semaphore) → display semaphore value
tid → using the coffee machine
tid → finished making coffee
signal (semaphore)
getValue (semaphore) → display semaphore value
exit (NULL)
```

INITIALISE (n)

```
sem semaphore
n = no. of people
pthread threads[n]
string array people[n]
sem_init (semaphore, 0, 1)
// 0 – semaphore to work with threads
// 1 – semaphore value is initially 1
for i=1 to n
        pthread_create (threads[i], COFFEE-BREAK, people[i])
for i=1 to n
        pthread_join (threads[i])
sem_destroy (semaphore)
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_05$ cc coffee.c
warspinix@Yuvaraj-Laptop:~/os/exp_05$ ./a.out
Krishnaa is waiting for coffee.
Yuvaraj is waiting for coffee.
Kabilesh is waiting for coffee.

Krishnaa is using the coffee machine.
Semaphore value while the coffee machine is being used = 0

Krishnaa has finished making coffee.
Semaphore value while the coffee machine is free = 1

Yuvaraj is using the coffee machine.
Semaphore value while the coffee machine is being used = 0

Yuvaraj has finished making coffee.
Semaphore value while the coffee machine is free = 1

Kabilesh is using the coffee machine.
Semaphore value while the coffee machine is being used = 0

Kabilesh has finished making coffee.
Semaphore value while the coffee machine is free = 1

All coffee breaks are over.
warspinix@Yuvaraj-Laptop:~/os/exp_05$
```

Evaluation Criteria	Observation	Record
Ability for problem definition and realisation	**10/10**	**/10**
Ability to design and analyse	**10/10**	**/10**
Ability to implement and validate	**10/10**	**/10**

RESULT

Various concepts pertaining to deadlocks were understood, and the coffee break synchronisation problem was successfully dealt with using semaphores.

Exp. No	7
Date	28/03/24

SIMULATION OF PREEMPTIVE AND NON-PREEPMTIVE PROCESS SCHEDULING ALGORITHMS

This experiment maps to the following COs and POs.

CO2	Use modern OS system calls and synchronisation libraries in software/hardware interfaces.
CO3	Understand the benefits of thread over process and implement synchronised programs using multithreading concepts.
PO1	Engineering knowledge
PO2	Problem analysis
PO3	Design/development of solutions
PO5	Modern tool usage
PO9	Individual and team work
PO10	Communication

AIM

To implement the following non-preemptive, preemptive, and real time process scheduling algorithms:

- First Come First Serve – FCFS
- Shortest Job First – SJF
- Priority – Non Pre-emptive
- Round Robin
- Shortest Remaining Time First – SRTF (OR) SJF Pre-emptive
- Priority – Pre-emptive
- Rate Monotonic Scheduling - RMS
- Earliest Deadline First – EDF

COMMON ALGORITHMS

INPUT: *List of jobs with arrival time AT, burst time BT, completion time CT*
OUTPUT: *Turnaround time TAT for each job + Average TAT*

TURNAROUND-TIME (jobs)

```
sum = 0
n = 0
for each job j
        j→TAT = j→CT – j→AT
        sum = sum + j→TAT
        n = n+1
return sum/n
```

INPUT: *list of jobs with AT, BT, CT, TAT*
OUTPUT: *waiting time WT for each job + Average WT*

WAITING-TIME (jobs)

```
sum = 0
n = 0
for each job j
        j→WT = j→TAT – j→BT
        sum = sum + j→WT
        n = n+1
return sum/n
```

FIRST COME FIRST SERVE

FCFS Scheduling is non-preemptive. Whichever process arrives first is scheduled and executed first. When a new process arrives, it can begin its execution only after the previous process has been fully completed.

ALGORITHM

INPUT: *List of jobs with AT, BT, and remaining time RT = BT*
OUTPUT: *FCFS scheduling order of jobs*

FCFS-SCHEDULER (jobs)

```
q = sorted (jobs) → AT
time = 0
while !q.empty()
        current = q.front()
        pid = current→id
        if (jobs→AT [pid] >= time )
                schedule current
                time = time+1
                jobs→RT [pid] --
                if (jobs→RT [pid] == 0)
                        jobs→CT [pid] = time
                        q.dequeue()
        else
                none
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_06/nonPreemptive$ cc fcfs.c
warspinix@Yuvaraj-Laptop:~/os/exp_06/nonPreemptive$ ./a.out
Process Details...
Process ID: 1
Process 1 AT = 0
Process 1 BT = 6
Process ID: 2
Process 2 AT = 0
Process 2 BT = 3
Process ID: 3
Process 3 AT = 0
Process 3 BT = 3

Scheduling Order

Current Time    Current Process
0               1
1               1
2               1
3               1
4               1
5               1
6               2
7               2
8               2
9               3
10              3
11              3

End of Schedule

PID     AT      BT      CT      TAT     WT
1       0       6       6       6       0
2       0       3       9       9       6
3       0       3       12      12      9

Average Turnaround Time = 9.00
Average Waiting Time = 5.00
warspinix@Yuvaraj-Laptop:~/os/exp_06/nonPreemptive$
```

SHORTEST JOB FIRST

This is also a non-preemptive scheduling algorithm. Here, the burst times of all the arrived jobs are compared, and whichever process has the shortest burst time is scheduled first. Once said process has been completed, the next process with the shortest burst time is scheduled.

ALGORITHM

INPUT: *List of jobs with AT, BT, and RT*
OUTPUT: *SJF scheduling order of jobs*

SJF-SCHEDULER (jobs)

```
q = sorted (jobs) → BT
time = 0
while !q.empty()
        current = q.front()
        pid = current→id
        if (jobs→AT [pid] >= time )
                schedule current
                time = time+1
                jobs→RT [pid] --
                if (jobs→RT [pid] == 0)
                        jobs→CT [pid] = time
                        q.dequeue()
        else
                none
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_06/nonPreemptive$ cc sjf.c
warspinix@Yuvaraj-Laptop:~/os/exp_06/nonPreemptive$ ./a.out
Receiving the Burst Times of Each Process...
Burst Time of Process 1: 6
Burst Time of Process 2: 8
Burst Time of Process 3: 7
Burst Time of Process 4: 3

Process Scheduling Order...

Process ID      Burst Time
4               3
1               6
3               7
2               8

PID     AT      BT      CT      TAT     WT
1       0       6       9       9       3
2       0       8       24      24      16
3       0       7       16      16      9
4       0       3       3       3       0

Average Turnaround Time = 13.00
Average Waiting Time = 7.00
warspinix@Yuvaraj-Laptop:~/os/exp_06/nonPreemptive$
```

PRIORITY – NON PREEMPTIVE

In non-preemptive priority scheduling, the priorities of all the currently arrived jobs are compared, and the job or process with the highest priority value is scheduled. Once this job is complete, the same comparison is repeated and subsequent processes are scheduled appropriately.

ALGORITHM

INPUT: *List of jobs with AT, BT, RT, and priority*
OUTPUT: *Non-preemptive priority scheduling order of jobs*

PRIORITY-NON-PREEMPTIVE-SCHEDULER (jobs)

```
q = sorted (jobs) → priority
time = 0
while !q.empty()
        current = q.front()
        pid = current→id
        if (jobs→AT [pid] >= time )
                schedule current
                time = time+1
                jobs→RT [pid] --
                if (jobs→RT [pid] == 0)
                        jobs→CT [pid] = time
                        q.dequeue()
        else
                none
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_06/nonPreemptive$ cc priority.c
warspinix@Yuvaraj-Laptop:~/os/exp_06/nonPreemptive$ ./a.out
Receiving Priorities and Burst Times of Each Process...
Burst Time of Process 1: 10
Priority of Process 1: 3
Burst Time of Process 2: 1
Priority of Process 2: 1
Burst Time of Process 3: 2
Priority of Process 3: 4
Burst Time of Process 4: 1
Priority of Process 4: 5
Burst Time of Process 5: 5
Priority of Process 5: 2

Process Scheduling Order...

Process ID      Priority
2               1
5               2
1               3
3               4
4               5

PID     AT      Priority        BT      CT      TAT     WT
1       0       3               10      16      16      6
2       0       1               1       1       1       0
3       0       4               2       18      18      16
4       0       5               1       19      19      18
5       0       2               5       6       6       1

Average Turnaround Time = 12.00
Average Waiting Time = 8.20
warspinix@Yuvaraj-Laptop:~/os/exp_06/nonPreemptive$
```

ROUND ROBIN

Round Robin scheduling can be considered as a preemptive FCFS that prevents the convoy effect (processes with shorter burst times waiting for the completion of processes with longer burst times). It introduces a concept called quantum time, which usually ranges from 10-100 ms. A particular process with a burst time higher than the quantum time will be allowed to execute for a time period equal to the assigned quantum time, after which it will be preempted by the next process in the queue. The same logic follows for the rest of the processes. This ensures a more uniform distribution of jobs.

ALGORITHM

INPUT: *Quantum Time QT, list of jobs with AT, BT, RT, and RQT = QT*
OUTPUT: *Round robin scheduling order of jobs*

ROUND-ROBIN-SCHEDULER (jobs, QT)

```
j = sorted (jobs)→AT
time = 0
while !j.empty()
        current = j.front()
        pid = current→id
        if (jobs→AT [pid] >= time )
                schedule current
                time = time+1
                jobs→RT [pid] --
                jobs→RQT [pid] --
                if (jobs→RQT [pid] == 0)
                        jobs→RQT [pid] = QT
                        j.dequeue()
                        j.enqueue(jobs[pid])
                if (jobs→RT [pid] == 0)
                        jobs→CT [pid] = time
                        j.dequeue()
        else
                none
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_06/preemptive$ cc roundRobin.c
warspinix@Yuvaraj-Laptop:~/os/exp_06/preemptive$ ./a.out
Quantum Time: 4
Arrival and Burst Times of Each Process
Process 1 AT = 0
Process 1 BT = 6
Process 2 AT = 0
Process 2 BT = 3
Process 3 AT = 0
Process 3 BT = 3

Scheduling Order

Current Time    Current Process
0               1
1               1
2               1
3               1
4               2
5               2
6               2
7               3
8               3
9               3
10              1
11              1

End of Schedule

PID     AT      BT      CT      TAT     WT
1       0       6       12      12      6
2       0       3       7       7       4
3       0       3       10      10      7

Average Turnaround Time = 9.67
Average Waiting Time = 5.67
warspinix@Yuvaraj-Laptop:~/os/exp_06/preemptive$
```

SHORTEST REMAINING TIME FIRST

SRTF can be considered as a preemptive SJF algorithm. At each instance of time, the remaining burst times of each of the arrived jobs are compared, and the job with the shortest remaining burst time is scheduled, pre-empted if necessary.

ALGORITHM

INPUT: *List of jobs with AT, BT, and RT*
OUTPUT: *SRTF scheduling order of jobs*

SRTF-SCHEDULER (jobs)

```
time = 0
no = count (jobs)
while no != 0
        min = INF
        for each job j
                if j→AT >= time AND j→CT == 0 AND j→RT < min
                        pid = j→PID
                        min = j→RT
        if pid
                schedule jobs→PID = pid
                time = time+1
                jobs→RT [pid] --
                if (jobs→RT [pid] == 0)
                        jobs→CT [pid] = time
                        no --
        else
                none
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_06/preemptive$ cc srtf.c
warspinix@Yuvaraj-Laptop:~/os/exp_06/preemptive$ ./a.out
Arrival and Burst Times of Each Process
Process 1 AT = 1
Process 1 BT = 2
Process 2 AT = 1
Process 2 BT = 1
Process 3 AT = 2
Process 3 BT = 3
Process 4 AT = 3
Process 4 BT = 2

Scheduling Order

Current Time    Current Process
0               None
1               2
2               1
3               1
4               4
5               4
6               3
7               3
8               3

End of Schedule

PID     AT      BT      CT      TAT     WT
1       1       2       4       3       1
2       1       1       2       1       0
3       2       3       9       7       4
4       3       2       6       3       1

Average Turnaround Time = 3.50
Average Waiting Time = 1.50
warspinix@Yuvaraj-Laptop:~/os/exp_06/preemptive$
```

PRIORITY – PREEMPTIVE

Priority preemptive scheduling compares the priorities of all the arrived processes at every instant and schedules the job with the highest priority, preempting if necessary.

ALGORITHM

INPUT: *List of jobs with AT, BT, RT, and priority*
OUTPUT: *SJF scheduling order of jobs*

PRIORITY-PREEMPTIVE-SCHEDULER (jobs)

```
time = 0
no = count (jobs)
while no != 0
        maxPriority = INF
        for each job j
                if j→AT >= time AND j→CT == 0
                        if j→priority < maxPriority
                                pid = j→PID
                                maxPriority = j→priority
        if pid
                schedule jobs→PID = pid
                time = time+1
                jobs→RT [pid] --
                if (jobs→RT [pid] == 0)
                        jobs→CT [pid] = time
                        no --
        else
                none
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_06/preemptive$ cc priority.c
warspinix@Yuvaraj-Laptop:~/os/exp_06/preemptive$ ./a.out
Arrival and Burst Times and Priority of Each Process
Process 1 AT = 0
Process 1 BT = 2
Priority of P1 = 4
Process 2 AT = 1
Process 2 BT = 1
Priority of P2 = 3
Process 3 AT = 2
Process 3 BT = 2
Priority of P3 = 2
Process 4 AT = 3
Process 4 BT = 1
Priority of P4 = 1

Scheduling Order

Current Time    Current Process
0               1
1               2
2               3
3               4
4               3
5               1

End of Schedule

PID     AT      BT      CT      TAT     WT
1       0       2       6       6       4
2       1       1       2       1       0
3       2       2       5       3       1
4       3       1       4       1       0

Average Turnaround Time = 2.75
Average Waiting Time = 1.25
warspinix@Yuvaraj-Laptop:~/os/exp_06/preemptive$
```

RATE MONOTONIC SCHEDULING

It is a preemptive, static, and priority-based algorithm. Given the periods, the priority is calculated as the inverse of the period. The job with the higher priority is scheduled first. If its period is complete, the next job is scheduled. But if the next job's period has begun before the current job's completion, then preemption takes place. The same cycle of jobs follows for every period.

ALGORITHM

INPUT: *total time, list of jobs with period P, BT, and RBT = BT*
OUTPUT: *RMS order of jobs up to the total time*

RMS-SCHEDULER (jobs, total time)

```
time = 0, tp = lcm (jobs→P)
while time <= total time
        if time mod tp == 0
                q = sorted (jobs) → 1/P
                no = count (jobs)
        if no == 0
                schedule none
        else
                current = q.front()
                pid = current→id
                for each job j in q upto nth job
                        if j→id != pid AND time mod j→P == 0
                                newid = j→id
                                while (pid != newid)
                                        q.dequeue()
                                        q.enqueue(jobs [pid])
                                        current = q.front()
                                        pid = current→id
                schedule current
                jobs[pid]→RBT --
                if (jobs[pid]→RBT == 0)
                        q.dequeue()
                        q.enqueue(current)
                        jobs[pid]→RBT == BT
                        no --
         time ++
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_06/realTime$ cc rms.c
warspinix@Yuvaraj-Laptop:~/os/exp_06/realTime$ ./a.out
Process Details
Process 1 Period = 10
Process 1 Burst Time = 4
Process 2 Period = 20
Process 2 Burst Time = 7

CPU Utilization = 0.75

Current Time    Current Process
0               1
1               1
2               1
3               1
4               2
5               2
6               2
7               2
8               2
9               2
10              1
11              1
12              1
13              1
14              2
15              None
16              None
17              None
18              None
19              None
warspinix@Yuvaraj-Laptop:~/os/exp_06/realTime$
```

EARLIEST DEADLINE FIRST

It is also a real-time algorithm that follows the concept of dynamic priority. Whichever job has the earliest deadline is scheduled, and preempted if necessary. That is, earlier the deadline, higher the priority of the process. This means that the priorities dynamically change with the execution of the processes.

ALGORITHM

INPUT: *total time, list of jobs with P, BT, RBT, and multiplier M=1*
OUTPUT: *EDF scheduling order of jobs up to the total time*

EDF-SCHEDULER (jobs, total time)

```
time = 0
q = sorted (jobs) → 1/P
while (time <= total time)
        current = q.front()
        pid = current→id
        deadline = (jobs→P * jobs→M) – time
        for each job j in q
                d = (j→P * j→M) - time
                if (d < deadline)
                        newid = j→id
                        deadline = d
                        exit
        while (newid != pid)
                current = q.front()
                pid = current→id
                q.dequeue()
                q.enqueue(current)
        schedule current
        jobs→RBT [pid] --
        if jobs→RBT [pid] == 0
                jobs→RBT [pid] == BT
                q.dequeue()
                q.enqueue(jobs[PID])
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_06/realTime$ cc edf.c
warspinix@Yuvaraj-Laptop:~/os/exp_06/realTime$ ./a.out
Process Details
Process 1 Period = 10
Process 1 Burst Time = 5
Process 2 Period = 16
Process 2 Burst Time = 7

CPU Utilization = 0.94

Current Time    Current Job
0               1
1               1
2               1
3               1
4               1
5               2
6               2
7               2
8               2
9               2
10              2
11              2
12              1
13              1
14              1
15              1
16              1
17              2
18              2
19              2
20              1
21              1
22              1
23              1
24              1
warspinix@Yuvaraj-Laptop:~/os/exp_06/realTime$
```

Evaluation Criteria	Observation	Record
Ability for problem definition and realisation	**9/10**	**/10**
Ability to design and analyse	**9/10**	**/10**
Ability to implement and validate	**9/10**	**/10**

RESULT

The various non-preemptive, preemptive, and real-time process scheduling algorithms were successfully implemented.

Exp. No	8
Date	04/04/24

SIMULATION OF FILE ALLOCATION STRATEGIES

This experiment maps to the following COs and POs.

CO5	Implement memory management schemes and page replacement schemes.
CO6	Simulate file allocation an organisation techniques.
PO1	Engineering knowledge
PO2	Problem analysis
PO3	Design/development of solutions
PO5	Modern tool usage
PO9	Individual and team work
PO10	Communication

AIM

To implement the following file allocation strategies:

- Contiguous Allocation
- Linked List Allocation
- Indexed Allocation

CONTIGUOUS ALLOCATION

In contiguous allocation, each file occupies a set of contiguous blocks. So, if the starting position of the file and the space it occupies is known, the file can be located, accessed, and modified easily.

It is the simplest memory allocation process, but also comes with many problems. Contiguous memory allocation can lead to external fragmentation, which is a scenario where the required space for a file is available in memory, but is located in different blocks in a non-contiguous manner such that allocation remains impossible.

ALGORITHM

INPUT: *memory, no. of files n, filename fn, starting position st, space sp*
OUTPUT: *Contiguous allocation of files*

CONTIGUOUS-ALLOCATION (memory, n)

```
a[memory]
for i = 1 to memory
        a[i]=0
for i = 1 to n
        read filename fn
        do: read st
        while: a[st] == 1
        do: read sp
        while: st+sp <= memory
        flag = 1
        for j = st to st+sp
                if a[j] == 1
                        flag = 0
                        exit
        if flag
                for j = st to st + sp
                        a[j] = 1
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_07$ g++ contig.cpp
warspinix@Yuvaraj-Laptop:~/os/exp_07$ ./a.out
Size of Memory = 100
Maximum No. of Files = 2

File Name: one
Starting Position (0 to 99): 20
Free Space Available = 80
No. of Units: 40
File one successfully loaded into memory units 20 to 59.
Press 1 to continue, 0 to exit: 1

File Name: two
Starting Position (0 to 99): 10
Free Space Available = 50
No. of Units: 30
No free memory.
Press 1 to continue, 0 to exit: 1

File Name: two
Starting Position (0 to 99): 30
This space is already occupied.
Starting Position (0 to 99): 60
Free Space Available = 40
No. of Units: 30
File two successfully loaded into memory units 60 to 89.
Maximum file count reached, exiting.

Memory Allocation Complete.

Available Files
-------------------------------------------------------
|No     |File Name      |Starting Position      |Space  |
|1      |one            |20                     |40     |
|2      |two            |60                     |30     |
-------------------------------------------------------
warspinix@Yuvaraj-Laptop:~/os/exp_07$
```

LINKED LIST ALLOCATION

Linked list allocation treats each file as a linked list of blocks. It avoids the problem of fragmentation by effectively using all the free blocks available due to the linked list property. However, linked list allocated causes performance lag as the file is scattered across many partially filed memory blocks.

ALGORITHM

INPUT: *memory, n, fn, st, sp*
OUTPUT: *Linked list allocation of files*

CLASS LINKED-LIST

NODE

```
no → node's position
next → link to next node
occupied → occupied status
nextNo → link to node where file is next located
file → file allocated to node
```

LINKED-LIST (memory)

```
root = NODE (1)
current = root
for i = 2 to memory
        new = NODE (i)
        current→next = new
        current = new
free = memory
```

GO-TO (pos)

```
current=&root
for i = 1 to memory
        if i == pos
                return
        current = current→next
```

OCCUPIED (pos)

```
GO-TO (pos)
return current→occupied
```

ALLOCATE (f, pos)

```
GO-TO (pos)
current→occupied = true
current→file = f
free --
```

GET-NEXT (cpos)

```
if cpos == memory: pos = 1
else: pos = cpos + 1
while
        GO-TO (pos)
        if current→occupied == false: return pos
        else: pos++
```

NEXT (cpos, npos)

```
GO-TO (cpos)
current→nextNo = npos
```

END OF CLASS

LINKED-LIST-ALLOCATION (memory, n)

```
LINKED-LIST a (memory)
for i = 1 to n
        read filename fn
        do: read starting position st
        while: a.OCCUPIED (st)
        do: read space sp
        while: sp > a.free
        a.ALLOCATE (fn, st)
        cloc = st
        for j = 2 to sp
                nloc = a.GET-NEXT (cloc)
                a.NEXT (cloc, nloc)
                cloc = nloc
                a.ALLOCATE (fn, cloc)
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_07$ g++ link.cpp
warspinix@Yuvaraj-Laptop:~/os/exp_07$ ./a.out
Size of Memory = 50
Maximum No. of Files = 3

File Name: one
Starting Position (1 to 50): 10
No. of Units: 20

Beginning memory allocation of one.
Unit 1 allocated at 10.
Unit 2 allocated at 11.
Unit 3 allocated at 12.
Unit 4 allocated at 13.
Unit 5 allocated at 14.
Unit 6 allocated at 15.
Unit 7 allocated at 16.
Unit 8 allocated at 17.
Unit 9 allocated at 18.
Unit 10 allocated at 19.
Unit 11 allocated at 20.
Unit 12 allocated at 21.
Unit 13 allocated at 22.
Unit 14 allocated at 23.
Unit 15 allocated at 24.
Unit 16 allocated at 25.
Unit 17 allocated at 26.
Unit 18 allocated at 27.
Unit 19 allocated at 28.
Unit 20 allocated at 29.
Memory allocation complete. one successfully loaded into memory.
Press 1 to continue, 0 to exit: 1
```

```
File Name: two
Starting Position (1 to 50): 15
This space is already occuupied.
Starting Position (1 to 50): 20
This space is already occuupied.
Starting Position (1 to 50): 5
No. of Units: 10

Beginning memory allocation of two.
Unit 1 allocated at 5.
Unit 2 allocated at 6.
Unit 3 allocated at 7.
Unit 4 allocated at 8.
Unit 5 allocated at 9.
Unit 6 allocated at 30.
Unit 7 allocated at 31.
Unit 8 allocated at 32.
Unit 9 allocated at 33.
Unit 10 allocated at 34.
Memory allocation complete. two successfully loaded into memory.
Press 1 to continue, 0 to exit: 1
```

```
File Name: three
Starting Position (1 to 50): 40
No. of Units: 15

Beginning memory allocation of three.
Unit 1 allocated at 40.
Unit 2 allocated at 41.
Unit 3 allocated at 42.
Unit 4 allocated at 43.
Unit 5 allocated at 44.
Unit 6 allocated at 45.
Unit 7 allocated at 46.
Unit 8 allocated at 47.
Unit 9 allocated at 48.
Unit 10 allocated at 49.
Unit 11 allocated at 50.
Unit 12 allocated at 1.
Unit 13 allocated at 2.
Unit 14 allocated at 3.
Unit 15 allocated at 4.
Memory allocation complete. three successfully loaded into memory.
Maximum file count reached, exiting.
```

```
|No     |Name          |Start  |End    |Total  |Memory Path
|1      |one           |10     |29     |20     |10->11->12->13->14->15->16->17->18->19->20->21->22->23->24->25->26->27->28->29
|2      |two           |5      |34     |10     |5->6->7->8->9->30->31->32->33->34
|3      |three         |40     |4      |15     |40->41->42->43->44->45->46->47->48->49->50->1->2->3->4
warspinix@Yuvaraj-Laptop:~/os/exp_07$
```

INDEXED ALLOCATION

Indexed allocation makes use of indexing or hashing in order to allocate a file to a particular index. Now there is extra information stored about the file alongside its starting position and space required, which is its index value. This makes accessing the file much, much faster.

ALGORITHM

INPUT: *memory, n, fn, sp, indices (no. of indices)*
OUTPUT: *Indexed allocation of files*

GET-INDEX-SIZE (indices, space)

```
a[indices][space]
for i = 1 to indices
    for j = 1 to space
        a[i][j]=0
```

FREE (index)

```
count = 0, start = 1st 0 space
for i = 1 to space
    if a[index][i]==0
        count ++
return start, count
```

HASH (name)

```
return name.length() % indices
```

INDEXED-ALLOCATION (memory, n)

```
for i = 1 to n
    read filename fn
    index = HASH (fn)
    read space sp
    if ! FREE (index) → st = start
        extend a → sp
    for i = st to st+sp
        a[index][i]=1
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_07$ g++ eindex.cpp
warspinix@Yuvaraj-Laptop:~/os/exp_07$ ./a.out
No. of Indices = 10
Space for Each Index = 50
No. of Files: 3

File Name: one
Space Required: 30
Allocating at index 3.
Press 0 to exit, anything else to continue: 1

File Name: three
Space Required: 20
Allocating at index 5.
Press 0 to exit, anything else to continue: 1

File Name: two
Space Required: 30
No free space available, so extending.
Allocating at index 3.

Maximum file count reached, exiting.

Indexed memory allocation complete.

-------------------------------------------------
|No      |Name            |Index   |Start   |Space   |
|1       |one             |3       |0       |30      |
|2       |three           |5       |0       |20      |
|3       |two             |3       |30      |30      |
-------------------------------------------------
warspinix@Yuvaraj-Laptop:~/os/exp_07$
```

Evaluation Criteria	Observation	Record
Ability for problem definition and realisation	**10/10**	**/10**
Ability to design and analyse	**10/10**	**/10**
Ability to implement and validate	**10/10**	**/10**

RESULT

The different file allocation strategies were successfully implemented.

Exp. No	9
Date	18/04/24

INTER PROCESS COMMUNICATION USING PIPES

This experiment maps to the following COs and POs.

CO3	Understand the benefits of thread over process and implement synchronised programs using multithreading concepts.
CO4	Analyse various IPC techniques in the operating system.
PO1	Engineering knowledge
PO2	Problem analysis
PO3	Design/development of solutions
PO5	Modern tool usage
PO9	Individual and team work
PO10	Communication

AIM

To implement inter process communication for a divide and conquer problem and enable sharing of resources between processes using unnamed pipes.

PIPES

Pipe is a connection between two processes, such that the standard output from one process becomes the standard input of the other process. It is a one-way communication, that is, one process can write to the pipe, while the other one reads from the pipe. Pipe is opened in an area of memory that is treated as a virtual file for the duration that it exists.

If a process tries to read from the pipe before something is written to it, the process is suspended until something is written.

If we call fork after creating a pipe, the parent and child processes can communicate via the pipe.

CHOSEN PROBLEM: BINARY SEARCH TREE

Insertion in binary search tree can be implemented using process control system calls. When an element is being inserted into an empty binary search tree, the parent process does the insertion by itself. When subsequent elements are inserted, based on whether the element goes to the left subtree or the right subtree, child processes are accordingly invoked, and these child processes insert the element at its correct position. At each level, the number of child processes grows by a power of 2. Each process has its own process ID, by which it is identified.

After the child process inserts the element at the required position, the updated tree is stored (OR) written into an unnamed pipe. The child process exits and returns to the parent process, at which point the parent process reads the unnamed pipe and receives the tree with the new element inserted at the correct position.

Thereby, with the help of unnamed pipes, unidirectional inter process communication can be implemented, where a child process performs the write operation on the pipe and the parent process performs the read operation on the pipe.

ALGORITHM

INPUT: *Binary Search Tree, data to be inserted*
OUTPUT*: Binary Search Tree with new data inserted by child processes that are created whenever the data moves to a left or right subtree, and the resultant tree shared between child and parent processes through pipes.*

BST-INSERT (root, data)

```
if root is NULL
        return new Node (data)
if data < root→data
        pipe_l[2]
        pid left = fork()
        if left
                root = BST-INSERT (root→left, data)
                write (pipe_l[1], root)
                close (pipe_l[1])
                exit()
         if parent
                wait (left, status, 0)
                if exited (status)
                        result = exit status
                        read (pipe_l[0], root)
                        close (pipe_l[0])
                        display left_pid, parent_pid details
else if data > root→data
        pipe_r[2]
        pid right = fork()
        if right
                root = BST-INSERT (root→right, data)
                write (pipe_l[1], root)
                close (pipe_l[1])
                exit()
        if parent
                wait (right, status, 0)
                if exited (status)
                        result = exit status
                        read (pipe_r[0], root)
                        close (pipe_r[0])
                        display right_pid, parent_pid details
return root
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_08$ cc pipe.c
warspinix@Yuvaraj-Laptop:~/os/exp_08$ ./a.out
Manually insert elements into the binary search tree.
Enter each element followed by Enter, type -1 to finish.
10
Parent Process with Process ID 26384 inserting 10.
Process with Process ID 26384 has created a node with value 10.
5
Parent Process with Process ID 26384 inserting 5.
Process with Process ID 26384 is calling its left child.
Child Process with Process ID 26391 inserting 5 to the left of 10.
Process with Process ID 26391 has created a node with value 5.
Child Process with Process ID 26391 has successfully inserted 5 and returned to its parent with Process ID 26384.
2
Parent Process with Process ID 26384 inserting 2.
Process with Process ID 26384 is calling its left child.
Child Process with Process ID 26394 inserting 2 to the left of 10.
Process with Process ID 26394 is calling its left child.
Child Process with Process ID 26395 inserting 2 to the left of 5.
Process with Process ID 26395 has created a node with value 2.
Child Process with Process ID 26395 has successfully inserted 2 and returned to its parent with Process ID 26394.
Child Process with Process ID 26394 has successfully inserted 2 and returned to its parent with Process ID 26384.
15
Parent Process with Process ID 26384 inserting 15.
Process with Process ID 26384 is calling its right child.
Child Process with Process ID 26408 inserting 15 to the right of 10.
Process with Process ID 26408 has created a node with value 15.
Child Process with Process ID 26408 has successfully inserted 15 and returned to its parent with Process ID 26384.
-1
Value to be Searched: 5
Parent Process with Process ID 26384 searching for key 5.
Child Process with Process ID 26431 searching for key 5 in the left subtree of 10.
Process with Process ID 26431 searching for key 5 in the root with value of 5.
Child Process with Process ID 26431 found the key 5 and has returned to its parent with Process ID 26384.
Parent Process with Process ID 26384 found the key 5 in the binary search tree.
Value to be Searched: -1

Inorder Traversal
2 5 10 15
warspinix@Yuvaraj-Laptop:~/os/exp_08$
```

Evaluation Criteria	Observation	Record
Ability for problem definition and realisation	**10/10**	**/10**
Ability to design and analyse	**10/10**	**/10**
Ability to implement and validate	**10/10**	**/10**

RESULT

Inter process communication was successfully implemented for binary search tree insertion with the help of unnamed pipes.

Exp. No	10
Date	18/04/24

INTER PROCESS COMMUNICATION USING FIFOs

This experiment maps to the following COs and POs.

CO3	Understand the benefits of thread over process and implement synchronised programs using multithreading concepts.
CO4	Analyse various IPC techniques in the operating system.
PO1	Engineering knowledge
PO2	Problem analysis
PO3	Design/development of solutions
PO5	Modern tool usage
PO9	Individual and team work
PO10	Communication

AIM

To implement inter process communication and resource sharing between two separate processes with named pipes/FIFO.

FIFO/NAMED PIPES

FIFO is an extension of the traditional pipe concept. A traditional, unnamed pipe lasts only as long as the process, whereas a named pipe or FIFO can last as long as the system is up, beyond the lifetime of the process.

FIFO appears as a file, and processes attach to this FIFO file for inter process communication. A FIFO file is a special kind of file created on the local storage, which allows two or more processes to communicate with each other by reading from/writing to this file.

Once a FIFO file has been created, any process can open it for reading or writing, in the same way as an ordinary file. However, it has to be open at both ends simultaneously before proceeding to do any input or output operations on it.

ALGORITHM

Program 1

INPUT: *Number n, choice (1 – prime, 2 – perfect, 3 – palindrome)*
OUTPUT: *Writes n and choice into FIFO, which is read by program 2; displays result of computation done by program 2 – whether n is prime or perfect or a palindrome – computation is done as per the choice of the user*

PASS-RECEIVE ()

```
mkfifo (fifo)
read n
read choice
pass [0], pass [1] = n, choice
f = open (fifo, WRITE)
write (f, pass)
close (f)
f = open (fifo, READ)
read (f, pass)
ans = pass[1]
close (f)
display ans
```

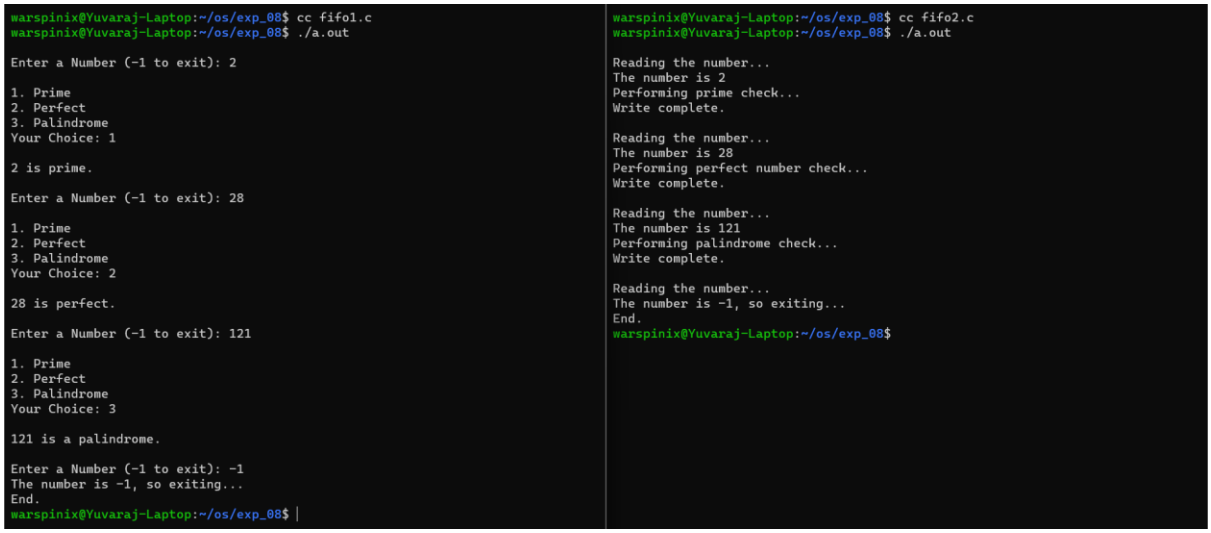
Program 2

INPUT: *pass[2]; pass[0] contains the number, pass[1] contains the choice*
OUTPUT: *Computes prime or perfect or palindrome check based on the choice, writes 1 into FIFO if true, 0 otherwise*

GET-COMPUTE ()

```
mkfifo (fifo)
f = open (fifo, READ)
read (f, pass)
choice = pass[1]
close (f)
if choice == 1
        pass [1] = prime(pass[0])
else if choice == 2
        pass [1] = perfect (pass[0])
else
        pass [1] = palindrome (pass[0])
f = open (fifo, WRITE)
write (f, pass)
close (f)
```

OUTPUT



```
warspinix@Yuvaraj-Laptop:~/os/exp_08$ cc fifo1.c
warspinix@Yuvaraj-Laptop:~/os/exp_08$ ./a.out

Enter a Number (-1 to exit): 2

1. Prime
2. Perfect
3. Palindrome
Your Choice: 1

2 is prime.

Enter a Number (-1 to exit): 28

1. Prime
2. Perfect
3. Palindrome
Your Choice: 2

28 is perfect.

Enter a Number (-1 to exit): 121

1. Prime
2. Perfect
3. Palindrome
Your Choice: 3

121 is a palindrome.

Enter a Number (-1 to exit): -1
The number is -1, so exiting...
End.
warspinix@Yuvaraj-Laptop:~/os/exp_08$
```

```
warspinix@Yuvaraj-Laptop:~/os/exp_08$ cc fifo2.c
warspinix@Yuvaraj-Laptop:~/os/exp_08$ ./a.out

Reading the number...
The number is 2
Performing prime check...
Write complete.

Reading the number...
The number is 28
Performing perfect number check...
Write complete.

Reading the number...
The number is 121
Performing palindrome check...
Write complete.

Reading the number...
The number is -1, so exiting...
End.
warspinix@Yuvaraj-Laptop:~/os/exp_08$
```

Evaluation Criteria	Observation	Record
Ability for problem definition and realisation	**/10**	**/10**
Ability to design and analyse	**/10**	**/10**
Ability to implement and validate	**/10**	**/10**

RESULT

Inter process communication was successfully implemented using named pipes/FIFO.

Exp. No	11
Date	25/04/24

IMPLEMENTATION OF CPU SCHEDULING POLICIES

This experiment maps to the following COs and POs.

CO6	Simulate file allocation and organisation techniques.
PO1	Engineering knowledge
PO2	Problem analysis
PO3	Design/development of solutions
PO5	Modern tool usage
PO9	Individual and team work
PO10	Communication

AIM

To implement the following CPU disk scheduling algorithms

- First Come First Serve – FCFS
- Shortest Seek Time First – SSTF
- SCAN/Elevator Scheduling
- C-SCAN – Circular SCAN
- LOOK
- C-LOOK – Circular LOOK

FIRST COME FIRST SERVE

In FCFS disk scheduling, the disk seeks are performed in the exact order that the request queue is received. This can be quite efficient or extremely inefficient based on the request queue received, making it a simple yet rather unreliable method of disk scheduling.

SHORTEST SEEK TIME FIRST

This disk scheduling algorithm selects the request with the minimum seek time from the current disk position. It may cause starvation of some requests in cases where there are a few outlier requests lying far away from all the other requests.

SCAN/ELEVATOR SCHEDULING

This algorithm simulates the working of an elevator. The disk arm starts from its head position, and moves either to the left side or right side of the disk, servicing requests till it goes to one end. Then, the disk movement is reversed and it goes on servicing the remaining requests on the other end till the last request is reached.

C-SCAN

In C-SCAN, the disk cylinders are treated as circular linked lists, meaning once an endpoint is reached, instead of reversing direction, the disk pointer continues in its circular direction, beginning in reverse order from the other end point. For example, if the total disk size is 200 (0 to 199), and if the disk begins from head pointer 50, moving towards 0 (servicing all the requests on its path), then instead of reversing its direction, it will go to 199 and service the remaining requests in reverse order.

LOOK

It is a version of SCAN, following the same principle, but instead of going entirely to the end point in a particular direction, it only goes as far as the last request in said direction, reversing immediately and servicing the remaining requests.

C-LOOK

It is a version of C-SCAN and the circular version of LOOK. Since we never really touch the end points in LOOK, the circular property works different in C-LOOK. Once the last request in a particular direction is reached, the disk pointer reverses direction and goes to the last request in the other direction, NOT servicing requests as it goes along. Then, it reverses direction and services the remaining requests.

CYLINDER MOVEMENTS

The efficiency and effectiveness of any disk scheduling algorithm is measured by the total number of cylinder movements made to service all the requests. Lower the number of cylinder movements, faster is the system and thus better is the disk scheduling algorithm.

ALGORITHM

INPUT: *disk to be scheduled, current disk c, previous disk p, head movements*
OUTPUT: *Schedule c, update movements*

SCHEDULE (disk, c, p, movements)

```
c = disk
schedule c
movements += abs (c – p)
p = c
```

INPUT: *disk size s, rq, head pointer h*
OUTPUT: *FCFS scheduling of disks, total head movements*

FCFS-SCHEDULER (s, rq, h)

```
movements = 0
p = h
for each request r in rq
        SCHEDULE (r, c, p, movements)
return movements
```

INPUT: *s, rq, h*
OUTPUT: *SSTF scheduling of disks, total head movements*

SSTF-SCHEDULER (s, rq, h)

```
movements = 0
p = h, i = 0, n = count (rq)
for i = 1 to n
        min = s
        cntr = 0
        for each request r in rq
                if abs (p – r) < min
                        min = abs (p - r)
                        pos = cntr
                cntr ++
        SCHEDULE (rq[pos], c, p, movements)
        rq [pos] = s
return movements
```

INPUT: *s, rq, h, direction d*
OUTPUT: *SCAN scheduling of disks, total head movements*

SCAN-SCHEDULER (s, rq, h, d)

```
movements = 0
p = h, disk = sorted (rq), n = count (rq)
lc = count (disk, requests < head)
if d == left
        for i = lc down to 1
                SCHEDULE (disk[i], c, p, movements)
        SCHEDULE (0, c, p, movements)
        for i = lc+1 to n
                SCHEDULE (disk[i], c, p, movements)
else
        for i = lc+1 to n
                SCHEDULE (disk[i], c, p, movements)
        SCHEDULE (s, c, p, movements)
        for i = lc down to 1
                SCHEDULE (disk[i], c, p, movements)
return movements
```

INPUT: *s, rq, h, direction d*
OUTPUT: *C-SCAN scheduling of disks, total head movements*

C-SCAN-SCHEDULER (s, rq, h, d)

```
movements = 0
p = h, disk = sorted (rq), n = count (rq)
lc = count (disk, requests < head)
if d == left
        for i = lc down to 1
                SCHEDULE (disk[i], c, p, movements)
        SCHEDULE (0, c, p, movements)
        c = s
        for i = n down to lc+1
                SCHEDULE (disk[i], c, p, movements)
else
        for i = lc+1 to n
                SCHEDULE (disk[i], c, p, movements)
        SCHEDULE (s, c, p, movements)
        c = 0
        for i = 1 to lc
                SCHEDULE (disk[i], c, p, movements)
return movements
```

INPUT: *s, rq, h, d*
OUTPUT: *LOOK scheduling of disks, total head movements*

LOOK-SCHEDULER (s, rq, h, d)

```
movements = 0
p = h, disk = sorted (rq), n = count (rq)
lc = count (disk, requests < head)
if d == left
        for i = lc down to 1
                SCHEDULE (disk[i], c, p, movements)
        for i = lc+1 to n
                SCHEDULE (disk[i], c, p, movements)
else
        for i = lc+1 to n
                SCHEDULE (disk[i], c, p, movements)
        for i = lc down to 1
                SCHEDULE (disk[i], c, p, movements)
return movements
```

INPUT: *s, rq, h, d*
OUTPUT: *C-LOOK scheduling of disks, total head movements*

C-LOOK-SCHEDULER (s, rq, h, d)

```
movements = 0
p = h, disk = sorted (rq), n = count (rq)
lc = count (disk, requests < head)
if d == left
        for i = lc down to 1
                SCHEDULE (disk[i], c, p, movements)
        for i = n down to lc+1
                SCHEDULE (disk[i], c, p, movements)
else
        for i = lc+1 to n
                SCHEDULE (disk[i], c, p, movements)
        for i = 1 to lc
                SCHEDULE (disk[i], c, p, movements)
return movements
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_09$ g++ final.cpp
warspinix@Yuvaraj-Laptop:~/os/exp_09$ ./a.out
Total size of disk: 200

Receiving request queue... (-1 to Exit)
Disk position: 14
Disk position: 37
Disk position: 65
Disk position: 67
Disk position: 98
Disk position: 122
Disk position: 124
Disk position: 183
Disk position: -1

Head pointer: 53

Request Queue: 14 37 65 67 98 122 124 183

0. Exit
1. FCFS
2. SSTF
3. SCAN
4. C-SCAN
5. LOOK
6. C-LOOK
Your Choice: 1

Commencing FCFS disk scheduling...
14 has been scheduled.
37 has been scheduled.
65 has been scheduled.
67 has been scheduled.
98 has been scheduled.
122 has been scheduled.
124 has been scheduled.
183 has been scheduled.
Disk scheduling complete.

Total cylinder movements = 208
```

```
Request Queue: 14 37 65 67 98 122 124 183

0. Exit
1. FCFS
2. SSTF
3. SCAN
4. C-SCAN
5. LOOK
6. C-LOOK
Your Choice: 2

Commencing SSTF disk scheduling...
65 has been scheduled.
67 has been scheduled.
37 has been scheduled.
14 has been scheduled.
98 has been scheduled.
122 has been scheduled.
124 has been scheduled.
183 has been scheduled.
Disk scheduling complete.

Total cylinder movements = 236

Request Queue: 14 37 65 67 98 122 124 183
```

```
Request Queue: 14 37 65 67 98 122 124 183

0. Exit
1. FCFS
2. SSTF
3. SCAN
4. C-SCAN
5. LOOK
6. C-LOOK
Your Choice: 3
Direction (0 - left, 1 - right): 0

Commencing SCAN disk scheduling...
37 has been scheduled.
14 has been scheduled.
Disk is at 0.
65 has been scheduled.
67 has been scheduled.
98 has been scheduled.
122 has been scheduled.
124 has been scheduled.
183 has been scheduled.
Disk scheduling complete.

Total cylinder movements = 236
```

```
Request Queue: 14 37 65 67 98 122 124 183

0. Exit
1. FCFS
2. SSTF
3. SCAN
4. C-SCAN
5. LOOK
6. C-LOOK
Your Choice: 4
Direction (0 - left, 1 - right): 1

Commencing C-SCAN disk scheduling...
65 has been scheduled.
67 has been scheduled.
98 has been scheduled.
122 has been scheduled.
124 has been scheduled.
183 has been scheduled.
Disk is at 199, and due to circular property, it is at 0.
14 has been scheduled.
37 has been scheduled.
Disk scheduling complete.

Total cylinder movements = 183
```

```
Request Queue: 14 37 65 67 98 122 124 183

0. Exit
1. FCFS
2. SSTF
3. SCAN
4. C-SCAN
5. LOOK
6. C-LOOK
Your Choice: 5
Direction (0 - left, 1 - right): 1

Commencing LOOK disk scheduling...
65 has been scheduled.
67 has been scheduled.
98 has been scheduled.
122 has been scheduled.
124 has been scheduled.
183 has been scheduled.
37 has been scheduled.
14 has been scheduled.
Disk scheduling complete.

Total cylinder movements = 299
```

```
Request Queue: 14 37 65 67 98 122 124 183

0. Exit
1. FCFS
2. SSTF
3. SCAN
4. C-SCAN
5. LOOK
6. C-LOOK
Your Choice: 6
Direction (0 - left, 1 - right): 0

Commencing C-LOOK disk scheduling...
37 has been scheduled.
14 has been scheduled.
183 has been scheduled.
124 has been scheduled.
122 has been scheduled.
98 has been scheduled.
67 has been scheduled.
65 has been scheduled.
Disk scheduling complete.

Total cylinder movements = 326

Request Queue: 14 37 65 67 98 122 124 183

0. Exit
1. FCFS
2. SSTF
3. SCAN
4. C-SCAN
5. LOOK
6. C-LOOK
Your Choice: 0
warspinix@Yuvaraj-Laptop:~/os/exp_09$
```

Evaluation Criteria	Observation	Record
Ability for problem definition and realisation	**10/10**	**/10**
Ability to design and analyse	**10/10**	**/10**
Ability to implement and validate	**10/10**	**/10**

RESULT

The various CPU disk scheduling algorithms were successfully implemented.

Exp. No	12
Date	02/05/24

IMPLEMENTATION OF MEMORY MANAGEMENT POLICIES

This experiment maps to the following COs and POs.

CO5	Implement memory management schemes and page replacement schemes.
CO6	Simulate file allocation and organisation techniques.
PO1	Engineering knowledge
PO2	Problem analysis
PO3	Design/development of solutions
PO5	Modern tool usage
PO9	Individual and team work
PO10	Communication

AIM

To simulate and implement the following memory management policies:

- First-Fit
- Best-Fit
- Worst-Fit

MEMORY ALLOCATION FOR PROCESSES

The memory size of the process that is to be allocated is compared with the memory space remaining in all the available blocks. Based on the choice of algorithm, a particular block is chosen and the process is allocated to that block. The strategies of some algorithms can be ineffective and end up not allocating any memory for some processes. It is thus important to choose the best strategy for memory allocation based on the blocks and processes.

FIRST-FIT

In this algorithm, the first block which has the memory required for the process is immediately allocated. It follows a greedy strategy – there might be a more optimal block that could possibly be allocated, but the strategy remains adamant and chooses whatever is the first available block that it encounters.

BEST-FIT

This is the most optimal algorithm in most cases. Here, the block which has the smallest remaining memory but can still allocate the given process is chosen.

WORST-FIT

In the worst-fit algorithm, the block with the maximum remaining memory that can allocate the given process is chosen.

ALGORITHM

INPUT: *process pr, block bl*
OUTPUT: *allocate pr to bl, reduce capacity of bl by size of pr*

ALLOCATE (pr, bl)

```
process[pr]→block = bl
block[bl]→processes.push(pid)
block[bl]→capacity -= process[pr] → size
```

INPUT: *'p' processes, 'b' blocks*
OUTPUT: *First fit memory allocation of processes to blocks*

FIRST-FIT (process, block)

```
flag = 0
for i = 1 to p
    for j = 1 to b
        if process[i]→size <= block[j]→capacity
            ALLOCATE (i, j)
            flag = 1
    if flag = 0
        none
```

INPUT: *'p' processes, 'b' blocks*
OUTPUT: *Best fit memory allocation of processes to blocks*

BEST-FIT (process, block)

```
flag = 0
for i = 1 to p
    min = INF, minpos = INF
    for j = 1 to b
        if process[i]→size <= block[j]→capacity
            if block[j]→capacity < min
                min = block[j]→capacity, minpos = j
                flag = 1
    if flag
        ALLOCATE (i, minpos)
    else
        none
```

INPUT: *'p' processes, 'b' blocks*
OUTPUT: *Worst fit memory allocation of processes to blocks*

WORST-FIT (process, block)

```
flag = 0
for i = 1 to p
        max = 0, maxpos = INF
        for j = 1 to b
                if process[i]→size <= block[j]→capacity
                        if block[j]→capacity > max
                                max = block[j]→capacity, maxpos = j
                                flag = 1
        if flag
                ALLOCATE (i, maxpos)
        else
                none
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/os/exp_10$ g++ final.cpp
warspinix@Yuvaraj-Laptop:~/os/exp_10$ ./a.out
No. of Processes = 4
No. of Blocks = 5

Processes
Memory required for process 1: 212
Memory required for process 2: 417
Memory required for process 3: 112
Memory required for process 4: 426

Blocks
Size of Block 1: 100
Size of Block 2: 500
Size of Block 3: 200
Size of Block 4: 300
Size of Block 5: 600

0. Exit
1. Best-Fit
2. First-Fit
3. Worst-Fit
Your Choice: 2

Beginning First-Fit Memory Allocation.
Allocating memory for process 1...
Process 1 has been allocated in block 2.
Allocating memory for process 2...
Process 2 has been allocated in block 5.
Allocating memory for process 3...
Process 3 has been allocated in block 2.
Allocating memory for process 4...
Process 4 can't be allocated.
Memory allocation complete.
```

```
Displaying block details...
-----------------------------------------------------
Block No.       Remaining Size  Allocated Processes
1               100
2               176             1 3
3               200
4               300
5               183             2
-----------------------------------------------------

Displaying process details...
-----------------------------------------------
Process ID      Required Size   Allocated Block
1               212             2
2               417             5
3               112             2
4               426             None
-----------------------------------------------
```

```
0. Exit
1. Best-Fit
2. First-Fit
3. Worst-Fit
Your Choice: 1

Beginning Best-Fit Memory Allocation.
Allocating memory for process 1...
Process 1 has been allocated in block 4.
Allocating memory for process 2...
Process 2 has been allocated in block 2.
Allocating memory for process 3...
Process 3 has been allocated in block 3.
Allocating memory for process 4...
Process 4 has been allocated in block 5.
Memory allocation complete.

Displaying block details...
-----------------------------------------------------
Block No.       Remaining Size  Allocated Processes
1               100
2               83              2
3               88              3
4               88              1
5               174             4
-----------------------------------------------------

Displaying process details...
-------------------------------------------------
Process ID      Required Size   Allocated Block
1               212             4
2               417             2
3               112             3
4               426             5
-------------------------------------------------
```

```
0. Exit
1. Best-Fit
2. First-Fit
3. Worst-Fit
Your Choice: 3

Beginning Memory Allocation.
Allocating memory for process 1...
Process 1 has been allocated in block 5.
Allocating memory for process 2...
Process 2 has been allocated in block 2.
Allocating memory for process 3...
Process 3 has been allocated in block 5.
Allocating memory for process 4...
Process 4 can't be allocated.
Memory allocation complete.

Displaying block details...
-----------------------------------------------------
Block No.       Remaining Size  Allocated Processes
1               100
2               83              2
3               200
4               300
5               276             1 3
-----------------------------------------------------

Displaying process details...
-------------------------------------------------
Process ID      Required Size   Allocated Block
1               212             5
2               417             2
3               112             5
4               426             None
-------------------------------------------------
```

```
0. Exit
1. Best-Fit
2. First-Fit
3. Worst-Fit
Your Choice: 0
warspinix@Yuvaraj-Laptop:~/os/exp_10$
```

Evaluation Criteria	Observation	Record
Ability for problem definition and realisation	**10/10**	**/10**
Ability to design and analyse	**10/10**	**/10**
Ability to implement and validate	**10/10**	**/10**

RESULT

The different memory management policies were successfully implemented.